

Java 1.0 — Primitive Types

Version: Java 1.0
Year: 1996
Feature: Primitive Types
Unique Class Name: PrimitiveTypes

1. Definition

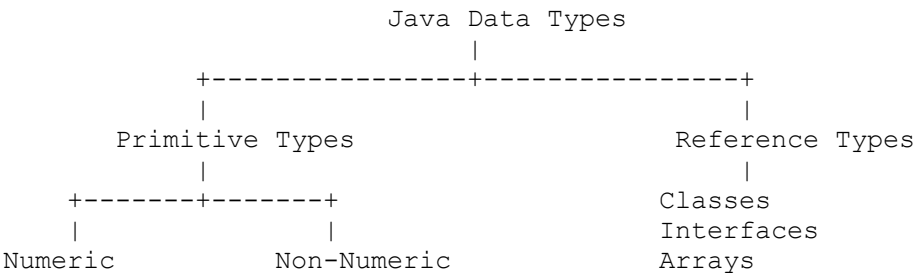
Primitive types are Java's built-in data types used to store simple, fundamental values directly. They are not objects and do not have methods of their own.

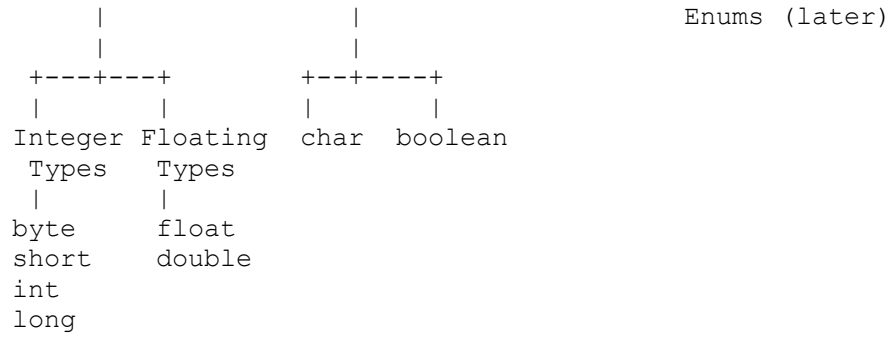
Java has **8 primitive data types**:

Type	Purpose	Example
byte	8-bit signed integer	byte age = 25;
short	16-bit signed integer	short marks = 500;
int	32-bit signed integer	int salary = 50000;
long	64-bit signed integer	long population = 1400000000L;
float	32-bit floating-point	float price = 99.5f;
double	64-bit floating-point	double pi = 3.14159;
char	16-bit Unicode character	char grade = 'A';
boolean	Logical value	boolean active = true;

Important: Java 1.0 introduced the primitive-type model, but some details of the Java type system and later APIs evolved in subsequent releases. The table above describes the standard Java primitive types.

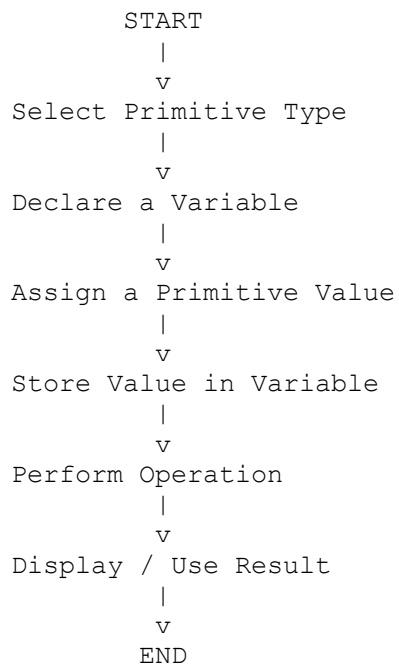
2. Primitive-Type Hierarchy



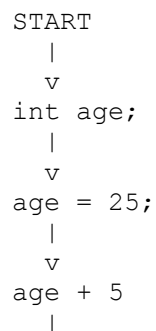


3. Flowchart

The following flow shows how a primitive variable is declared, assigned, and used:



Example flow



```
      v
30
  |
  v
Display 30
  |
  v
END
```

4. Java Code Example

```
public class PrimitiveTypes {

    public static void main(String[] args) {

        // Integer primitive types
        byte byteValue = 100;
        short shortValue = 1000;
        int intValue = 100000;
        long longValue = 100000000000L;

        // Floating-point primitive types
        float floatValue = 10.5f;
        double doubleValue = 20.123456;

        // Character primitive type
        char charValue = 'A';

        // Boolean primitive type
        boolean booleanValue = true;

        // Display values
        System.out.println("byte      : " + byteValue);
        System.out.println("short     : " + shortValue);
        System.out.println("int      : " + intValue);
        System.out.println("long     : " + longValue);
        System.out.println("float    : " + floatValue);
        System.out.println("double   : " + doubleValue);
        System.out.println("char     : " + charValue);
        System.out.println("boolean  : " + booleanValue);
    }
}
```

Output

```
byte      : 100
short     : 1000
int       : 100000
long      : 100000000000
float     : 10.5
double    : 20.123456
```

```
char    : A
boolean : true
```

5. How the Code Works

For example:

```
int intValue = 100000;
```

This consists of:

int	intValue	=	100000;
Type	Variable	Assignment	Value

int tells Java that intValue can hold a 32-bit signed integer.

Similarly:

```
char charValue = 'A';
```

means:

```
char → variable type
charValue → variable name
'A' → character value
```

And:

```
boolean booleanValue = true;
```

stores a logical value:

true / false

Key point

Primitive types form the **foundation of Java's type system**. They provide efficient storage for simple values such as numbers, characters, and logical states, while **reference types** are used to refer to objects and other complex data structures.

Java 1.0 — Reference Types

Version: Java 1.0

Year: 1996

Feature: Reference Types

Unique Class Name: ReferenceTypes

1. Definition

A **reference type** in Java is a data type whose variable stores a **reference to an object or array**, rather than directly storing the object's data itself.

Reference types include:

- **Class types**
- **Interface types**
- **Array types**

A reference variable can refer to an object, refer to an array, or contain null when it does not currently refer to an object.

For example:

```
String name = "Anurag";
```

Conceptually:

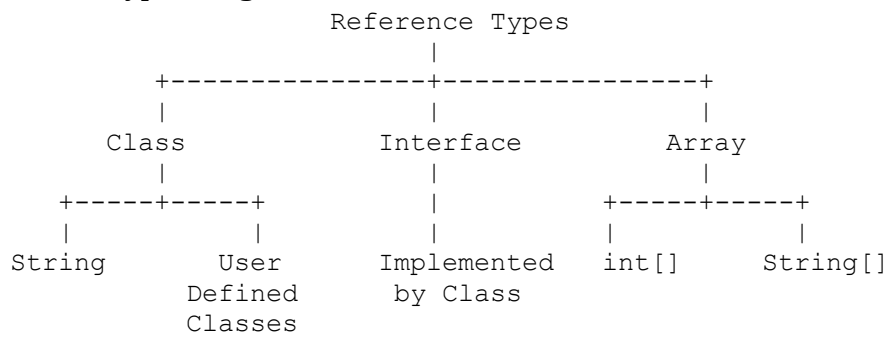
```
name
|
| reference
v
+-----+
| String object |
| "Anurag"      |
+-----+
```

Unlike a primitive variable such as:

```
int age = 25;
```

where the variable directly represents the primitive value, a reference variable identifies an object.

2. Reference-Type Categories



Main categories

Reference Type	Example	Meaning
Class	String	Reference to a class object
User-defined class	Student	Reference to an instance of your class
Interface	Runnable	Reference whose actual object implements the interface
Array	int[]	Reference to an array object
null	Student s = null;	Reference points to no object

3. Reference Variable vs Object

Consider:

```
Student student = new Student();
```

There are two important things here:

```
Student student = new Student();
|               |
|               +---- Object created
|               |
+----- Reference variable
```

Conceptually:

Stack / Reference

Heap / Object

student

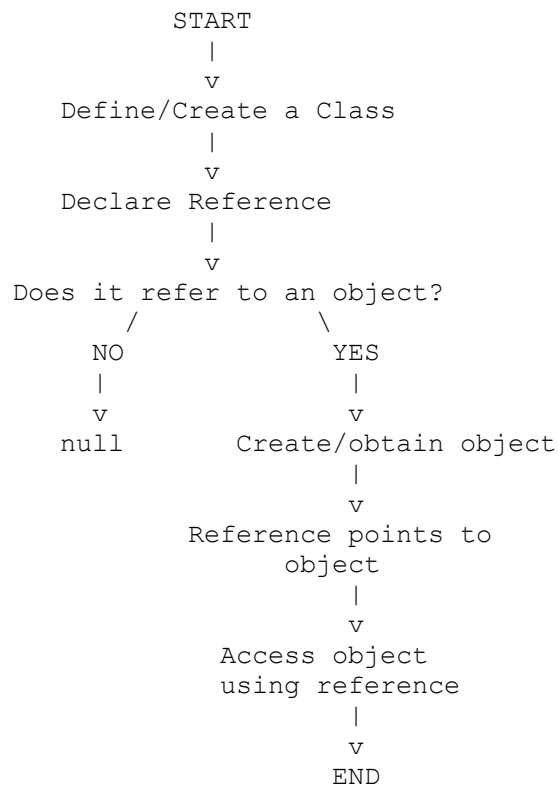
|
| reference
v

```
+-----+  
| Student object |  
| name = "Anurag" |  
| age  = 25        |  
+-----+
```

The exact physical memory layout is JVM-dependent, so this diagram is a conceptual model rather than a specification of where every value must reside.

4. Flowchart

Creating and using a reference type



Example

```
START
  |
  v
Student student
  |
  v
new Student()
  |
  v
Reference stored in
student variable
  |
  v
student.name = "Anurag"
  |
  v
student.display()
  |
  v
END
```

5. Java Code Example

```
public class ReferenceTypes {

    // User-defined class
    static class Student {

        String name;
        int age;

        void display() {
            System.out.println("Name : " + name);
            System.out.println("Age : " + age);
        }
    }

    public static void main(String[] args) {

        // 1. Class reference type
        Student student = new Student();

        student.name = "Anurag";
        student.age = 25;

        System.out.println("Student Object:");
        student.display();

        // 2. String reference type
```

```

        String message = "Hello Java";

        System.out.println("\nString Object:");
        System.out.println(message);

        // 3. Array reference type
        int[] numbers = {10, 20, 30, 40, 50};

        System.out.println("\nArray Object:");
        for (int number : numbers) {
            System.out.println(number);
        }

        // 4. null reference
        Student anotherStudent = null;

        System.out.println("\nNull Reference:");
        System.out.println(anotherStudent);
    }
}

```

Output

Student Object:

Name : Anurag
Age : 25

String Object:

Hello Java

Array Object:

10
20
30
40
50

Null Reference:

null

6. Understanding Each Example

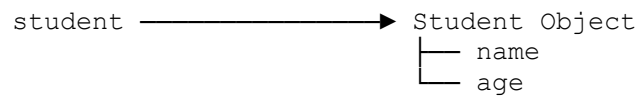
A. User-defined class

Student student = new Student();

Here:

- Student → reference type
- student → reference variable
- new Student() → creates an object
- = → assigns the object's reference to student

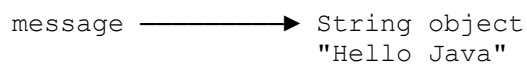
Conceptually:



B. String

```
String message = "Hello Java";
```

String is a **class type**, so message is a reference variable.

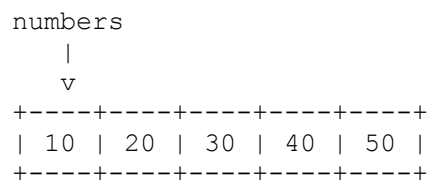


C. Array

```
int[] numbers = {10, 20, 30, 40, 50};
```

int[] is an **array reference type**.

Conceptually:



Notice that int itself is a primitive type, but int[] is a **reference type**.

D. null

```
Student anotherStudent = null;
```

This means the reference currently does not refer to any object.

```
anotherStudent
  |
  X
no object
```

If you try:

```
anotherStudent.display();
```

you will get a `NullPointerException` at runtime.

7. Primitive vs Reference Types

Feature	Primitive Type	Reference Type
Represents	Simple value	Reference to an object/array
Examples	int, double, char, boolean	String, Student, int[]
Can be null?	No	Yes
Has object identity?	No	Object referred to can have identity
Can call methods directly?	No	Yes, through a non-null reference
Example	int age = 25;	Student s = new Student();

Simple comparison

```
int age = 25;
```

Conceptually:

age —► 25

Whereas:

```
Student s = new Student();
```

Conceptually:

s —————▶ Student object

8. Important Concept: Multiple References

Two reference variables can refer to the **same object**:

```
Student s1 = new Student();
```

```
s1.name = "Anurag";
```

```
Student s2 = s1;
```

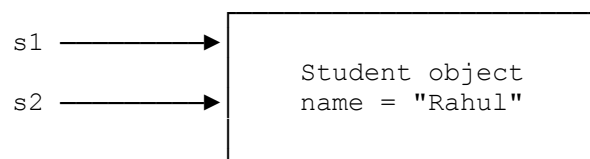
```
s2.name = "Rahul";
```

```
System.out.println(s1.name);
```

Output:

Rahul

Why?



s1 and s2 are two references pointing to the **same object**.

9. Core Definition to Remember

A reference type is a Java type whose variable holds a reference to an object or array rather than directly representing a primitive value.

The three fundamental reference-type categories to remember are:

Reference Types

|

```
+— Class
|
+— Interface
|
+— Array
```

And the most important distinction is:

Primitive

↓

stores a value

Reference

↓

refers to an object/array

This distinction is fundamental to understanding **objects, inheritance, polymorphism, arrays, interfaces, garbage collection, casting, null, and Java memory behavior.**

Java 1.0 — Operators

Java Version: 1.0

Year: 1996

Feature: Operators

Unique Class Name: Operators

1. Definition

An **operator** in Java is a special symbol or keyword that tells the Java compiler to perform an operation on one or more **operands**.

For example:

```
int result = 10 + 20;
```

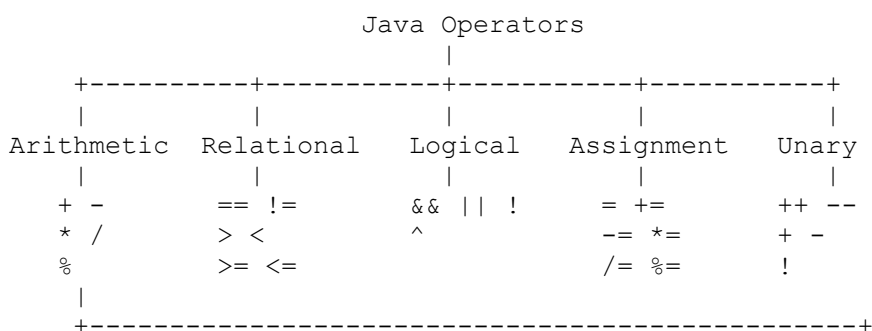
Here:

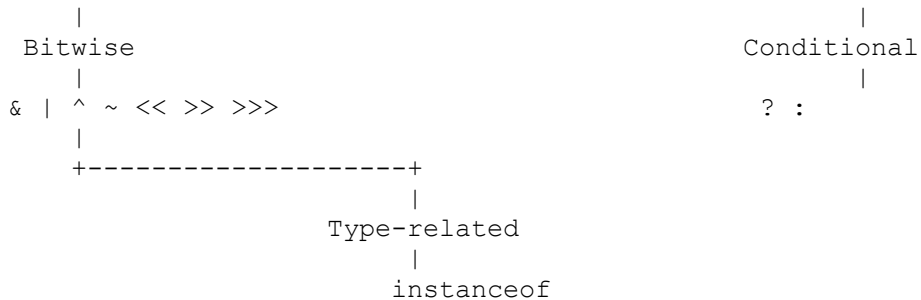
10	+	20	=	30
Operand	Operator	Operand		Result

Operators are fundamental to Java because they allow programs to perform:

- Arithmetic calculations
 - Comparisons
 - Logical decisions
 - Assignments
 - Bit-level operations
 - Conditional expressions
 - Type checking
-

2. Java Operators — Main Categories



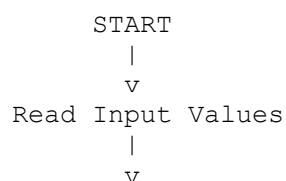


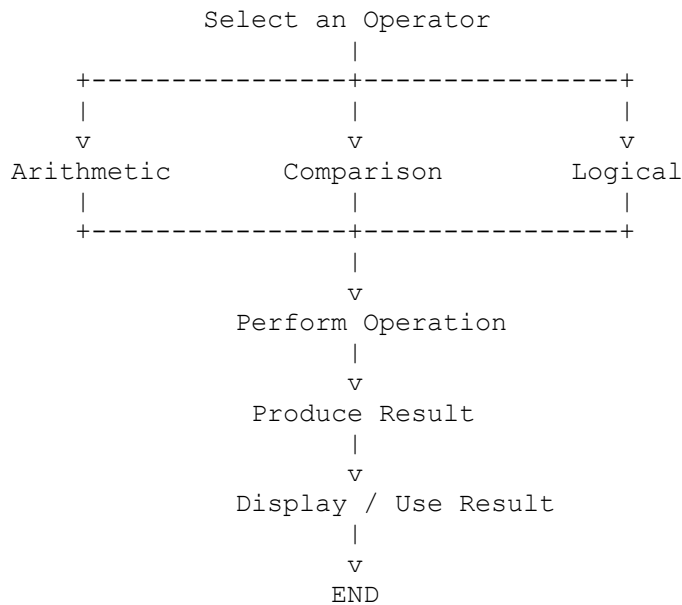
3. Important Operator Categories

Category	Operators	Purpose
Arithmetic	+ - * / %	Mathematical calculations
Unary	+ - ++ -- ! ~	Operate on one operand
Relational	== != > < >= <=	Compare values
Logical	&& !	Combine logical conditions
Assignment	= += -= *= /= %=	Assign/update values
Bitwise	& ^ ~	Perform bit-level operations
Shift	<< >> >>>	Shift bits
Conditional	?:	Select one of two expressions
Type comparison	instanceof	Test an object's type relationship

The exact operator set and language behavior have evolved across Java releases. The above represents the core Java operator families relevant to the Java 1.x language.

4. Flowchart





5. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Remainder	10 % 3	1

Example:

```
int a = 10;
int b = 3;
```

```
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a / b); // 3
System.out.println(a % b); // 1
```

Notice that:

10 / 3

produces 3 when both operands are int, because integer division discards the fractional part.

6. Relational Operators

Relational operators compare two values and produce a boolean result.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
int a = 10;  
int b = 20;
```

```
System.out.println(a == b); // false  
System.out.println(a != b); // true  
System.out.println(a < b);  // true  
System.out.println(a > b);  // false
```

7. Logical Operators

Logical operators operate on boolean expressions.

Operator	Name	Example
----------	------	---------

&&	Logical AND	a && b
!	Logical NOT	!a

Example:

```
int age = 25;
boolean citizen = true;

boolean eligible = age >= 18 && citizen;

System.out.println(eligible);
```

Output:

true

The && operator requires **both conditions** to be true.

8. Assignment Operators

Assignment operators assign values to variables.

Operator	Example	Equivalent
=	a = 10	Assign 10
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

Example:

```
int number = 10;
```

```
number += 5;
```

```
System.out.println(number);
```

Output:

15

9. Unary Operators

A unary operator operates on **one operand**.

Important unary operators include:

+	Unary plus
-	Unary minus
++	Increment
--	Decrement
!	Logical NOT
~	Bitwise complement

Example:

```
int number = 10;
```

```
number++;
```

```
System.out.println(number);
```

Output:

11

Prefix vs postfix

```
int a = 10;
```

```
int x = ++a;
```

a is incremented first.

```
a = 11
```

```
x = 11
```

Whereas:

```
int b = 10;
```

```
int y = b++;
```

uses the original value first.

```
b = 11
```

```
y = 10
```

10. Bitwise Operators

Bitwise operators work with the binary representation of integral values.

&	AND
	OR
^	XOR
~	Complement

Example:

```
int a = 5; // 0101
```

```
int b = 3; // 0011
```

```
System.out.println(a & b);
```

Binary operation:

```
  0101
& 0011
-----
  0001
```

Therefore:

1

is produced.

11. Shift Operators

Java provides bit-shift operators:

<< Left shift
>> Signed right shift
>>> Unsigned right shift

Example:

```
int number = 8;
```

```
System.out.println(number << 1);
```

Conceptually:

```
8 = 00001000
```

```
8 << 1
```

```
00010000
```

```
↓
```

```
16
```

Output:

```
16
```

12. Conditional Operator

The conditional operator is:

?:

It is commonly called the **ternary operator** because it has three operands.

Syntax:

```
condition ? expression1 : expression2
```

Example:

```
int age = 20;
```

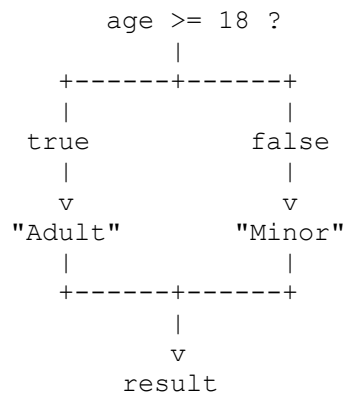
```
String result = age >= 18 ? "Adult" : "Minor";
```

```
System.out.println(result);
```

Output:

Adult

Flow:



13. instanceof

`instanceof` is used to test whether an object reference is compatible with a specified type.

Example:

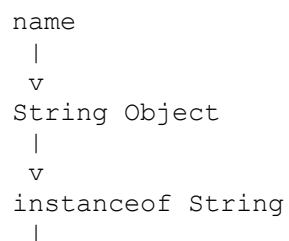
```
String name = "Java";
```

```
System.out.println(name instanceof String);
```

Output:

true

Conceptually:



v
true

14. Complete Java Code Example

```
public class Operators {

    public static void main(String[] args) {

        int a = 10;
        int b = 3;

        // 1. Arithmetic Operators
        System.out.println("=== Arithmetic Operators ===");

        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b));
        System.out.println("a % b = " + (a % b));

        // 2. Relational Operators
        System.out.println("\n=== Relational Operators ===");

        System.out.println("a == b : " + (a == b));
        System.out.println("a != b : " + (a != b));
        System.out.println("a > b : " + (a > b));
        System.out.println("a < b : " + (a < b));
        System.out.println("a >= b : " + (a >= b));
        System.out.println("a <= b : " + (a <= b));

        // 3. Logical Operators
        System.out.println("\n=== Logical Operators ===");

        boolean x = true;
        boolean y = false;

        System.out.println("x && y : " + (x && y));
        System.out.println("x || y : " + (x || y));
        System.out.println("!x : " + (!x));

        // 4. Assignment Operators
        System.out.println("\n=== Assignment Operators ===");

        int number = 10;

        number += 5;
        System.out.println("number += 5 : " + number);
```

```

number -= 3;
System.out.println("number -= 3 : " + number);

number *= 2;
System.out.println("number *= 2 : " + number);

number /= 4;
System.out.println("number /= 4 : " + number);

number %= 3;
System.out.println("number %= 3 : " + number);

// 5. Unary Operators
System.out.println("\n=== Unary Operators ===");

int value = 10;

System.out.println("value      : " + value);
System.out.println("++value    : " + (++value));
System.out.println("value++    : " + (value++));
System.out.println("value      : " + value);
System.out.println("--value    : " + (--value));

// 6. Bitwise Operators
System.out.println("\n=== Bitwise Operators ===");

int p = 5;
int q = 3;

System.out.println("p & q : " + (p & q));
System.out.println("p | q : " + (p | q));
System.out.println("p ^ q : " + (p ^ q));
System.out.println("~p   : " + (~p));

// 7. Shift Operators
System.out.println("\n=== Shift Operators ===");

int n = 8;

System.out.println("n << 1 : " + (n << 1));
System.out.println("n >> 1 : " + (n >> 1));
System.out.println("n >>> 1: " + (n >>> 1));

// 8. Conditional/Ternary Operator
System.out.println("\n=== Conditional Operator ===");

int age = 20;

String status =
    age >= 18 ? "Adult" : "Minor";

System.out.println("Status : " + status);

```

```

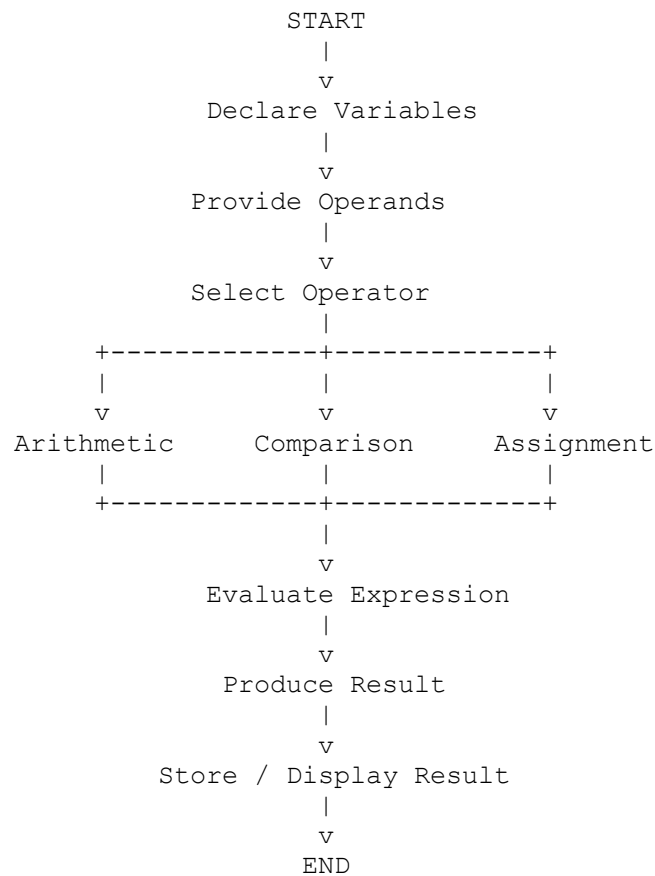
// 9. instanceof Operator
System.out.println("\n=== instanceof Operator ===");

String text = "Java";

System.out.println(
    "text instanceof String : "
    + (text instanceof String)
);
}
}

```

15. Overall Operator Execution Flow



16. Key Concept

The simplest way to remember Java operators is:

OPERAND + OPERATOR + OPERAND

For example:

10 + 20

means:

10	+	20
Operand	Operator	Operand
	Addition	
	v	
	30	

So, **operators are the symbols/keywords that tell Java what operation to perform on operands.** They are one of the fundamental building blocks of Java expressions, together with **variables, literals, types, and method calls.**

Java 1.0 — if-else

Java Version: 1.0

Year: 1996

Feature: if-else

Unique Class Name: IfElse

1. Definition

The **if-else statement** is a Java **selection/decision-making statement** used to execute different blocks of code depending on whether a condition evaluates to true or false.

Basic syntax:

```
if (condition) {  
    // executed when condition is true  
} else {  
    // executed when condition is false  
}
```

The condition must evaluate to a **boolean value** (true or false).

Simple example

```
int age = 20;  
  
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Since age >= 18 is true, the if block executes.

2. Components of if-else

```
if (condition) {  
    statement;  
} else {  
    statement;  
}
```

Component	Meaning
if	Java keyword for decision making

(condition)	Boolean expression being evaluated
{ }	Block of statements
else	Executes when the if condition is false
statement	Code that should execute

Conceptually:

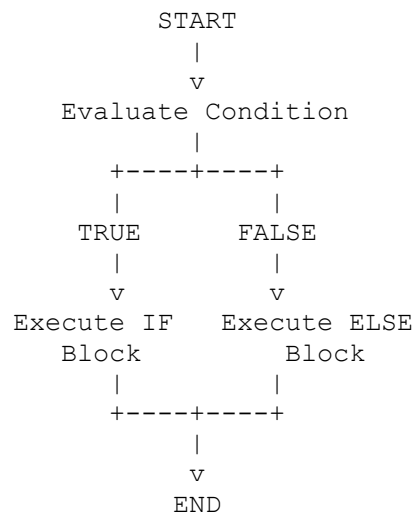
```

if
|
+---- condition
      |
      true/false
      |
+-----+-----+
|               |
true          false
|             |
v             v
if-block     else-block

```

3. Flowchart

Basic if-else flowchart



Example flow

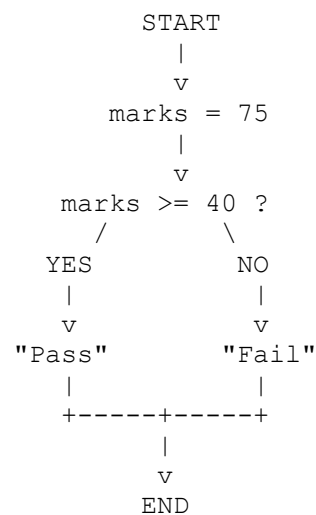
Suppose:

```
int marks = 75;
```

and:

```
if (marks >= 40) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

Flow:



Because $75 \geq 40$ is true, "Pass" is printed.

4. Java Code Example

```
public class IfElse {  
  
    public static void main(String[] args) {  
  
        int marks = 75;  
  
        if (marks >= 40) {  
            System.out.println("Student has passed.");  
        }  
    }  
}
```

```
    } else {  
        System.out.println("Student has failed.");  
    }  
}  
}
```

Output

Student has passed.

5. How the Code Executes

The expression:

`marks >= 40`

is evaluated first.

With:

`marks = 75`

Java evaluates:

`75 >= 40`

Result:

`true`

Therefore:

`System.out.println("Student has passed.");`

is executed.

The else block is skipped.

```
marks = 75  
  |  
  v  
75 >= 40  
  |  
  v  
TRUE
```

```
    |
    v
    "Student has passed."
```

6. if Without else

The else part is optional.

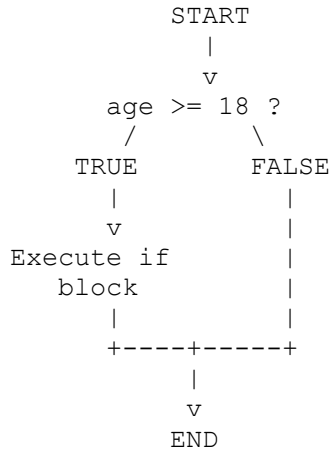
```
public class IfElse {

    public static void main(String[] args) {

        int age = 20;

        if (age >= 18) {
            System.out.println("Eligible to vote.");
        }
    }
}
```

If the condition is false, nothing inside the if block executes.



7. if-else if-else

When there are **multiple conditions**, Java allows an else-if chain.

Syntax:

```
if (condition1) {
```

```

} else if (condition2) {

} else if (condition3) {

} else {

}

```

Example:

```

public class IfElse {

    public static void main(String[] args) {

        int marks = 85;

        if (marks >= 90) {
            System.out.println("Grade A+");
        } else if (marks >= 80) {
            System.out.println("Grade A");
        } else if (marks >= 70) {
            System.out.println("Grade B");
        } else if (marks >= 60) {
            System.out.println("Grade C");
        } else {
            System.out.println("Fail");
        }

    }

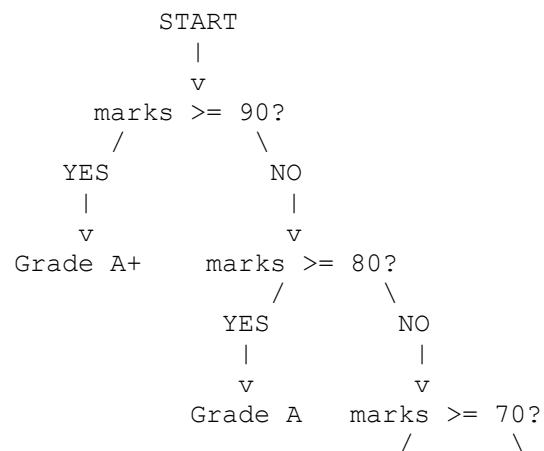
}

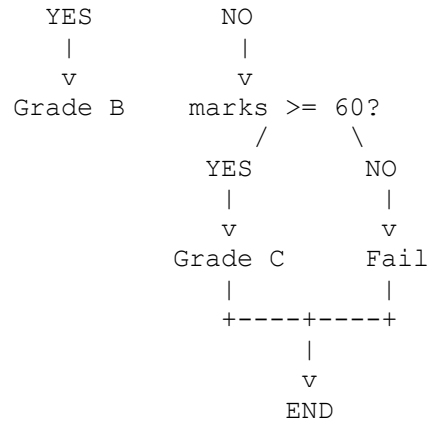
```

Output:

Grade A

Flowchart





Important: In an if-else-if chain, Java executes the **first matching condition** and then skips the remaining else-if and else blocks.

8. Nested if-else

An if-else statement can be placed inside another if or else block. This is called a **nested if**.

Example:

```

public class IfElse {

    public static void main(String[] args) {

        int age = 25;
        boolean hasLicense = true;

        if (age >= 18) {

            if (hasLicense) {
                System.out.println("You can drive.");
            } else {
                System.out.println("You need a driving license.");
            }

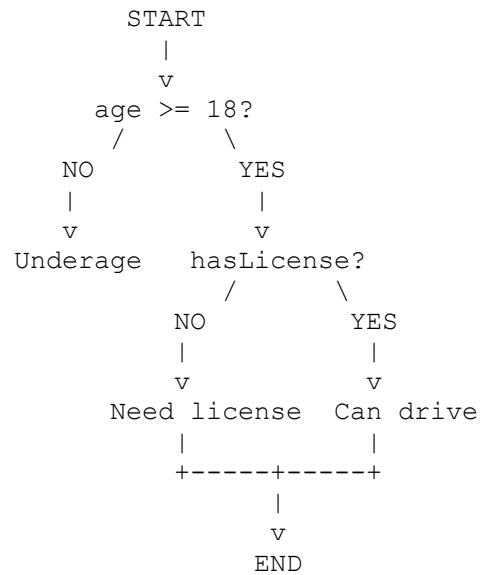
        } else {
            System.out.println("You are underage.");
        }
    }
}

```

Output:

You can drive.

Flow:



9. Using Logical Operators with if-else

Conditions can be combined using operators such as:

- && — AND
- || — OR
- ! — NOT

Example:

```
public class IfElse {

    public static void main(String[] args) {

        int age = 25;
        boolean citizen = true;

        if (age >= 18 && citizen) {
            System.out.println("Eligible.");
        } else {
            System.out.println("Not eligible.");
        }
    }
}
```

Here:

`age >= 18 && citizen`

means:

```
age >= 18    → true
citizen      → true
              |
              v
          true && true
              |
              v
              true
```

Output:

Eligible.

10. Important Rule: Boolean Condition

Java requires the if condition to be a boolean expression.

Correct:

```
int age = 20;

if (age >= 18) {
    System.out.println("Adult");
}
```

Incorrect:

```
int age = 20;

if (age) {
    System.out.println("Adult");
}
```

Unlike languages such as C/C++, Java does **not** treat an integer such as 20 as a boolean condition.

11. if-else vs switch

Both can be used for decision making, but they are suited to different situations.

if-else	Switch
Good for conditions and ranges	Good for selecting among discrete cases
Supports complex boolean expressions	Uses matching cases
Supports >, <, >=, &&, etc.	Traditionally based on equality/case matching
Can handle ranges naturally	Less convenient for ranges
Example: marks >= 40	Example: day == 1

Example where if-else is appropriate:

```
if (marks >= 40) {  
    System.out.println("Pass");  
} else {  
    System.out.println("Fail");  
}
```

12. Complete Demonstration Program

```
public class IfElse {  
  
    public static void main(String[] args) {  
  
        int marks = 75;  
        int age = 20;  
        boolean citizen = true;  
  
        // Simple if-else  
        if (marks >= 40) {  
            System.out.println("Result: Pass");  
        } else {  
            System.out.println("Result: Fail");  
        }  
  
        // if-else-if ladder  
        if (marks >= 90) {
```

```

        System.out.println("Grade: A+");
    } else if (marks >= 80) {
        System.out.println("Grade: A");
    } else if (marks >= 70) {
        System.out.println("Grade: B");
    } else if (marks >= 60) {
        System.out.println("Grade: C");
    } else {
        System.out.println("Grade: Fail");
    }

    // Logical condition
    if (age >= 18 && citizen) {
        System.out.println("Status: Eligible");
    } else {
        System.out.println("Status: Not Eligible");
    }
}
}

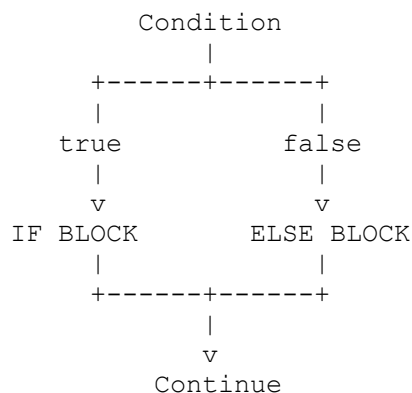
```

Output

Result : Pass
 Grade : B
 Status : Eligible

13. Core Execution Model

The entire if-else concept can be remembered as:



Key definition

if-else is a Java selection statement that evaluates a boolean condition and executes the if block when the condition is true; otherwise, it executes the else block.

It is one of the fundamental **control-flow statements** in Java and forms the basis for implementing decisions such as validation, authorization, grading, eligibility checks, business rules, and conditional processing.

Java 1.0 — switch Statement

Java Version: 1.0

Year: 1996

Feature: switch statement

Unique Class Name: SwitchStatement

1. Definition

The **switch statement** is a Java **selection/decision-making statement** used to execute one block of code from multiple possible alternatives based on the value of an expression.

Instead of writing many if-else if conditions, switch can make code easier to read when comparing one expression against several discrete values.

Basic syntax

```
switch (expression) {  
  
    case value1:  
        // statements  
        break;  
  
    case value2:  
        // statements  
        break;  
  
    default:  
        // statements  
}
```

For example:

```
int day = 2;  
  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;
```

```
case 2:
    System.out.println("Tuesday");
    break;

default:
    System.out.println("Invalid day");
}
```

Output:

Tuesday

2. Main Components of switch

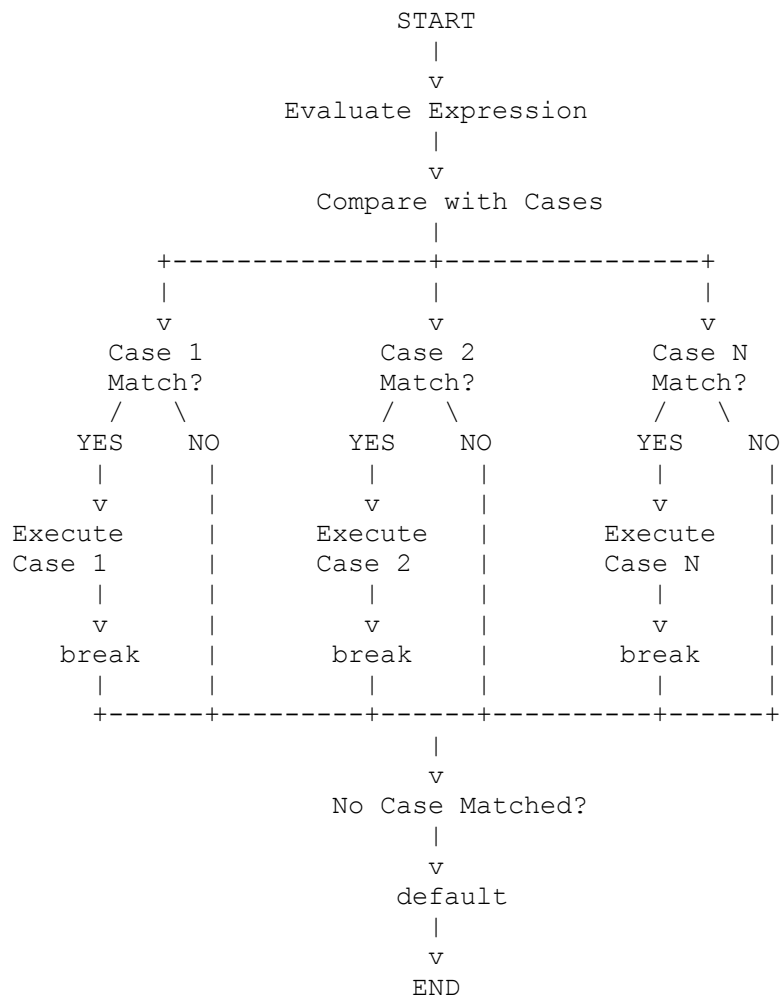
```
switch (expression) {

    case value:
        statement;
        break;

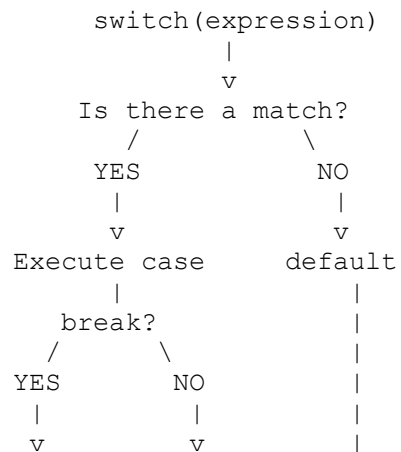
    default:
        statement;
}
```

Component	Meaning
switch	Java keyword that begins the selection statement
expression	Value that Java evaluates
case	Defines a possible matching value
break	Terminates the current switch
default	Executes when no case matches
:	Separates a case value from its statements

3. switch Flowchart



A simpler conceptual flow is:



```

END      Next case      |
          |              |
          +-----+
          |
          v
          END

```

4. Simple Java Example

```

public class SwitchStatement {

    public static void main(String[] args) {

        int day = 2;

        switch (day) {

            case 1:
                System.out.println("Monday");
                break;

            case 2:
                System.out.println("Tuesday");
                break;

            case 3:
                System.out.println("Wednesday");
                break;

            default:
                System.out.println("Invalid day");

        }

    }

}

```

Output

Tuesday

Execution

The variable contains:

day = 2;

Java evaluates:

switch (day)

Then checks:

`2 == 1 → false`

`2 == 2 → true`

Therefore:

`System.out.println("Tuesday");`

executes.

Then:

`break;`

terminates the switch.

5. Why break Is Important

Consider:

```
int day = 2;

switch (day) {

    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Invalid");
}
```

Because `day == 2`, Java enters case 2.

```
case 2
|
v
Print Tuesday
```

```
    |  
    v  
break  
    |  
    v  
Exit switch
```

Without break:

```
int day = 2;  
  
switch (day) {  
  
    case 1:  
        System.out.println("Monday");  
        break;  
  
    case 2:  
        System.out.println("Tuesday");  
  
    case 3:  
        System.out.println("Wednesday");  
        break;  
}
```

Output:

```
Tuesday  
Wednesday
```

This behavior is called **fall-through**.

6. Fall-Through Behavior

When a matching case does not contain break, execution continues into the following cases until a break, the end of the switch, or another control-flow transfer is reached.

Example:

```
public class SwitchStatement {  
  
    public static void main(String[] args) {  
  
        int number = 1;  
  
        switch (number) {
```

```

        case 1:
            System.out.println("One");

        case 2:
            System.out.println("Two");

        case 3:
            System.out.println("Three");
            break;

        default:
            System.out.println("Other");
    }
}

```

Output:

```

One
Two
Three

```

Flow:

```

number = 1
  |
  v
case 1 matched
  |
  v
Print One
  |
  v
No break
  |
  v
case 2
  |
  v
Print Two
  |
  v
No break
  |
  v
case 3
  |
  v
Print Three
  |
  v
break

```

|
v
Exit switch

7. default Case

The default case executes when none of the case labels matches the switch expression.

Example:

```
public class SwitchStatement {  
    public static void main(String[] args) {  
        int day = 8;  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            default:  
                System.out.println("Invalid day");  
        }  
    }  
}
```

Output:

Invalid day

Flow:

```
day = 8  
|  
v  
case 1? → NO  
|  
v  
case 2? → NO  
|
```

```
    v
case 3? → NO
    |
    v
default
    |
    v
"Invalid day"
```

The default clause is optional, but it is often useful for handling unexpected values.

8. Multiple Cases With the Same Code

Multiple cases can intentionally share one block.

Example:

```
public class SwitchStatement {

    public static void main(String[] args) {

        int day = 6;

        switch (day) {

            case 1:
            case 2:
            case 3:
            case 4:
            case 5:
                System.out.println("Weekday");
                break;

            case 6:
            case 7:
                System.out.println("Weekend");
                break;

            default:
                System.out.println("Invalid day");

        }

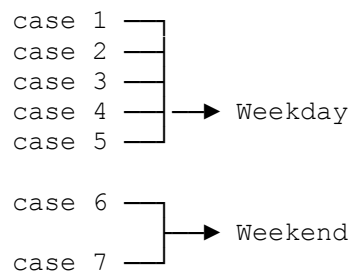
    }

}
```

Output:

Weekend

Conceptually:



This is one of the useful traditional patterns of the switch statement.

9. switch With char

Traditional Java switch also supports integral-compatible primitive types such as char.

Example:

```
public class SwitchStatement {  
    public static void main(String[] args) {  
        char grade = 'A';  
        switch (grade) {  
            case 'A':  
                System.out.println("Excellent");  
                break;  
            case 'B':  
                System.out.println("Very Good");  
                break;  
            case 'C':  
                System.out.println("Good");  
                break;  
            default:  
                System.out.println("Needs Improvement");  
        }  
    }  
}
```

Output:

Excellent

10. switch With byte, short, and int

Example:

```
public class SwitchStatement {  
    public static void main(String[] args) {  
        int option = 2;  
        switch (option) {  
            case 1:  
                System.out.println("Create");  
                break;  
            case 2:  
                System.out.println("Update");  
                break;  
            case 3:  
                System.out.println("Delete");  
                break;  
            default:  
                System.out.println("Invalid option");  
        }  
    }  
}
```

Output:

Update

11. switch vs if-else

switch	if-else
Good for multiple discrete alternatives	Good for general conditions

Compares an expression against cases	Evaluates boolean expressions
Uses case	Uses conditions
Traditionally uses break to prevent fall-through	Does not have switch-style fall-through
Good for menu selections	Good for ranges and complex conditions
Example: day == 1, day == 2	Example: marks >= 40

switch example

```
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
}
```

if-else example

```
if (marks >= 90) {
    System.out.println("A+");
} else if (marks >= 80) {
    System.out.println("A");
} else {
    System.out.println("B or below");
}
```

A switch is generally more natural when selecting among **discrete values**, whereas if-else is generally more natural for **ranges or compound conditions**.

12. Complete Java Code Example

```
public class SwitchStatement {

    public static void main(String[] args) {

        int choice = 3;

        System.out.println("Menu:");

        switch (choice) {
```

```

        case 1:
            System.out.println("1. Create User");
            break;

        case 2:
            System.out.println("2. Update User");
            break;

        case 3:
            System.out.println("3. Delete User");
            break;

        case 4:
            System.out.println("4. Search User");
            break;

        case 5:
            System.out.println("5. Exit");
            break;

        default:
            System.out.println("Invalid choice");
    }
}
}

```

Output

Menu:
3. Delete User

Execution flow

```

START
|
v
choice = 3
|
v
switch(choice)
|
v
case 1?
|
NO
|
v
case 2?
|
NO
|
v
case 3?

```

```

      |
    YES
      |
      v
"Delete User"
      |
      v
    break
      |
      v
    END

```

13. Important switch Rules

Rule 1 — Case values must be appropriate constant labels

```

case 1:
case 2:
case 3:

```

Each case represents a possible match.

Rule 2 — Duplicate case values are not allowed

This is invalid:

```

switch (number) {

    case 1:
        System.out.println("One");
        break;

    case 1:
        System.out.println("Another One");
        break;
}

```

The compiler reports a duplicate case label.

Rule 3 — default is optional

Both are valid:

```

switch (number) {
    case 1:
        // ...
        break;
}

```

```
}
```

and:

```
switch (number) {  
    case 1:  
        // ...  
        break;  
  
    default:  
        // ...  
}
```

Rule 4 — break is optional syntactically

But without it, normal switch execution can **fall through** to subsequent cases.

Rule 5 — case does not itself create a separate scope

Braces can be introduced when separate local-variable scopes are needed.

14. Historical Note

For **Java 1.0 (1996)**, the original switch statement was much more limited than the modern Java switch feature.

The early form centered on integral-compatible selection, such as:

```
byte  
short  
char  
int
```

Later Java releases expanded switch considerably. For example:

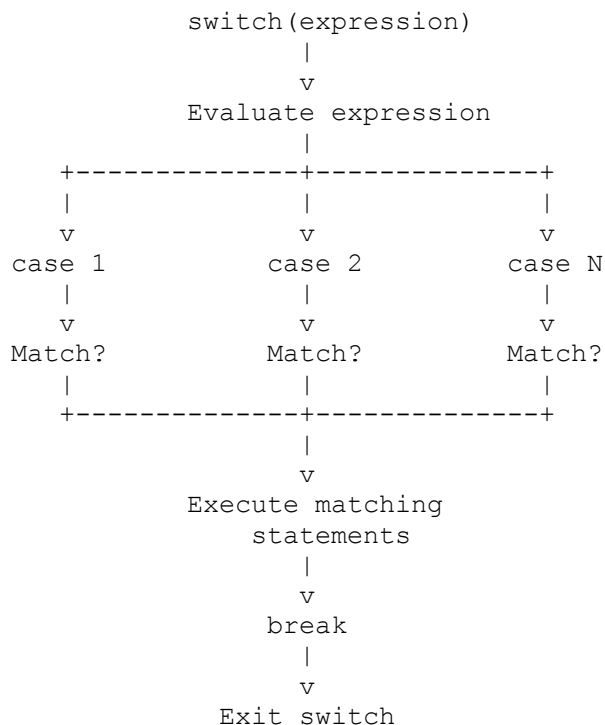
```
Java 1.x  
↓  
Traditional switch/case/break  
↓  
Java 5  
↓  
enum support  
↓  
Java 7  
↓  
String support  
↓
```

Java 14
↓
switch expressions
↓
Later Java
↓
Modern switch enhancements

Therefore, when documenting **Java 1.0 specifically**, it is important not to attribute modern switch features such as String cases, enum cases, arrow labels (case X ->), or yield to Java 1.0.

15. Core Concept

The complete concept can be remembered as:



Key definition

The switch statement is a Java selection statement that evaluates an expression and transfers control to the statements associated with a matching case label; if no case matches, the optional default block is used.

The most important concepts to remember are:

switch → expression → case → match → statements → break → exit.

Java 1.0 — for Loop

Java Version: 1.0

Year: 1996

Feature: for loop

Unique Class Name: ForLoop

1. Definition

The **for loop** is an **iteration/repetition statement** in Java used to execute a block of code repeatedly while a specified condition remains true.

It is especially useful when the number of iterations is known or can be controlled by a counter.

Basic syntax

```
for (initialization; condition; update) {  
    // statements to repeat  
}
```

Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

2. Components of a for Loop

```
for (initialization; condition; update) {  
    statement;  
}
```

Each part has a specific purpose:

Component	Purpose	Example
-----------	---------	---------

Initialization	Executes once before the loop begins	int i = 1
Condition	Determines whether another iteration should occur	i <= 5
Update	Changes the loop-control variable	i++
Body	Code executed during each iteration	System.out.println(i)

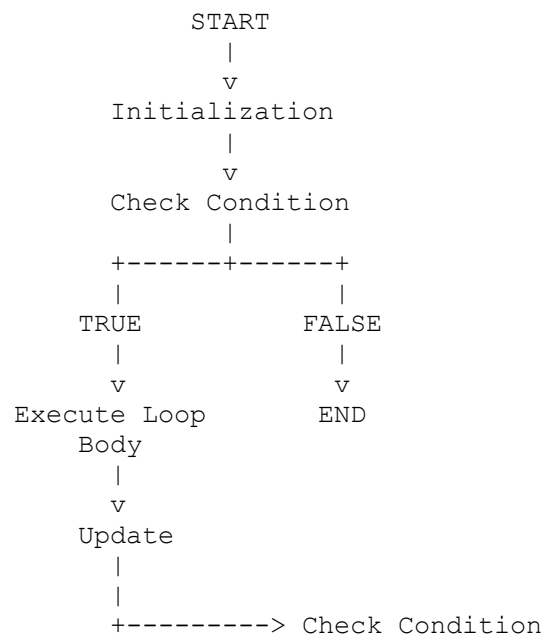
Conceptually:

```

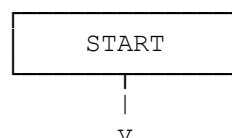
for ( initialization ; condition ; update )
    |
    |
    |
    +----- Change counter
    +----- Check condition
    +----- Initialize counter

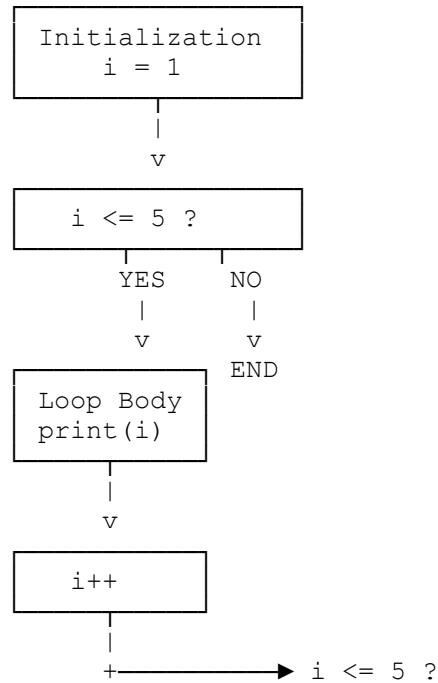
```

3. Basic Flowchart



Complete flow





4. Java Code Example

```
public class ForLoop {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Iteration: " + i);  
        }  
    }  
}
```

Output

```
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4  
Iteration: 5
```

5. Step-by-Step Execution

Consider:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Step 1 — Initialization

```
int i = 1;
```

This happens **only once**.

```
i = 1
```

Step 2 — Condition

```
i <= 5
```

Java checks:

```
1 <= 5 → true
```

Therefore, the loop body executes.

Step 3 — Body

```
System.out.println(i);
```

Output:

```
1
```

Step 4 — Update

```
i++;
```

Now:

```
i = 2
```

Step 5 — Condition again

```
2 <= 5 → true
```

The process repeats.

6. Complete Iteration Table

Iteration	i before condition	Condition $i \leq 5$	Output	i++
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	4	true	4	5
5	5	true	5	6
6	6	false	—	—

When i becomes 6:

$6 \leq 5 \rightarrow \text{false}$

The loop terminates.

7. Execution Model

The fundamental execution order is:

Initialization



Condition



true?



YES



NO



Body



END



Update

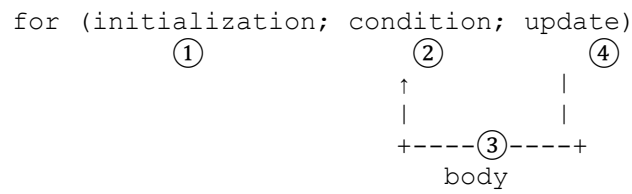


Condition

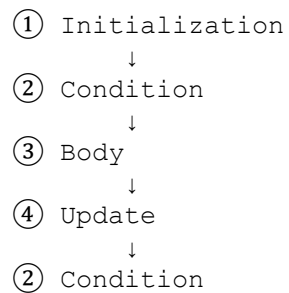


(repeat)

Therefore:



More precisely:



8. for Loop With Multiple Initialization Variables

Java allows multiple expressions in the initialization and update sections.

Example:

```
public class ForLoop {

    public static void main(String[] args) {

        for (int i = 1, j = 10; i <= 5; i++, j--) {
            System.out.println("i = " + i + ", j = " + j);
        }
    }
}
```

Output:

```
i = 1, j = 10
i = 2, j = 9
i = 3, j = 8
i = 4, j = 7
i = 5, j = 6
```

Here:

```
int i = 1, j = 10
```

is initialization, while:

```
i++, j--
```

is the update section.

9. for Loop Without Initialization

The initialization section can be omitted.

```
public class ForLoop {  
    public static void main(String[] args) {  
        int i = 1;  
        for (; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

Notice the first semicolon:

```
for (; i <= 5; i++)
```

The variable was initialized before the loop.

10. for Loop Without Update

The update can also be omitted.

```
public class ForLoop {  
    public static void main(String[] args) {
```

```

int i = 1;

for (; i <= 5;) {

    System.out.println(i);

    i++;
}
}

```

This works because the update is performed inside the loop body.

11. Infinite for Loop

If all three sections are omitted:

```

for (;;) {
    System.out.println("Running...");
}

```

the loop has no terminating condition and is therefore an **infinite loop**, unless something inside the body terminates it.

Flow:

```

START
|
v
for (;;)
|
v
Execute body
|
v
Repeat forever

```

A controlled infinite loop can be terminated with break:

```

for (;;) {

    System.out.println("Running");

    break;
}

```

12. for Loop With an if Statement

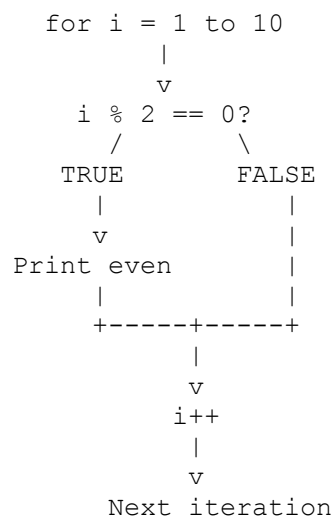
A for loop is often combined with conditional logic.

```
public class ForLoop {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i % 2 == 0) {  
                System.out.println(i + " is even");  
            }  
        }  
    }  
}
```

Output:

```
2 is even  
4 is even  
6 is even  
8 is even  
10 is even
```

Flow:



13. Nested for Loops

A for loop can be placed inside another for loop. This is called a **nested loop**.

Example:

```
public class ForLoop {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                System.out.println(  
                    "i = " + i + ", j = " + j  
                );  
            }  
        }  
    }  
}
```

Output:

```
i = 1, j = 1  
i = 1, j = 2  
i = 1, j = 3  
i = 2, j = 1  
i = 2, j = 2  
i = 2, j = 3  
i = 3, j = 1  
i = 3, j = 2  
i = 3, j = 3
```

Nested-loop flow

```
Outer Loop  
  |  
  v  
i = 1  
  |  
  v  
Inner Loop  
  |  
  +-- j = 1  
  +-- j = 2  
  +-- j = 3  
  |  
  v  
Outer Update  
  |  
  v  
i = 2  
  |
```

```

      v
Inner Loop
  |
  +-- j = 1
  +-- j = 2
  +-- j = 3
  |
  ...

```

For every one iteration of the outer loop, the inner loop completes all of its iterations.

14. for Loop With break

break immediately terminates the loop.

```

public class ForLoop {

    public static void main(String[] args) {

        for (int i = 1; i <= 10; i++) {

            if (i == 6) {
                break;
            }

            System.out.println(i);
        }
    }
}

```

Output:

```

1
2
3
4
5

```

Flow:

```

i == 6?
|
+---- TRUE ----> break ----> END
|
FALSE
|
v

```

```
Print i
|
v
i++
```

15. for Loop With continue

continue skips the remaining statements of the current iteration and proceeds to the next iteration.

```
public class ForLoop {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue;
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
4
5
```

When `i == 3`, the print statement is skipped.

16. for vs while vs do-while

Feature	for	while	do-while
Type	Entry-controlled	Entry-controlled	Exit-controlled
Initialization	Usually in loop header	Usually before loop	Usually before loop
Update	Usually in loop	Usually in body	Usually in body

	header		
Minimum executions	0	0	1
Best suited for	Counter-controlled repetition	Condition-controlled repetition	At-least-once execution
Basic form	for (...)	while (...)	do {...} while (...)

For example, this:

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

is conceptually similar to:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

17. Complete Java Code Example

```
public class ForLoop {

    public static void main(String[] args) {

        System.out.println("=== Basic For Loop ===");

        for (int i = 1; i <= 5; i++) {
            System.out.println("Iteration: " + i);
        }

        System.out.println("\n=== Even Numbers ===");

        for (int i = 1; i <= 10; i++) {

            if (i % 2 == 0) {
                System.out.println(i);
            }

        }

    }

}
```

```

System.out.println("\n=== Multiple Variables ===");

for (int i = 1, j = 5; i <= 5; i++, j--) {
    System.out.println(
        "i = " + i + ", j = " + j
    );
}

System.out.println("\n=== Nested For Loop ===");

for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(
            "i = " + i + ", j = " + j
        );
    }
}

System.out.println("\n=== Break ===");

for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break;
    }

    System.out.println(i);
}

System.out.println("\n=== Continue ===");

for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }

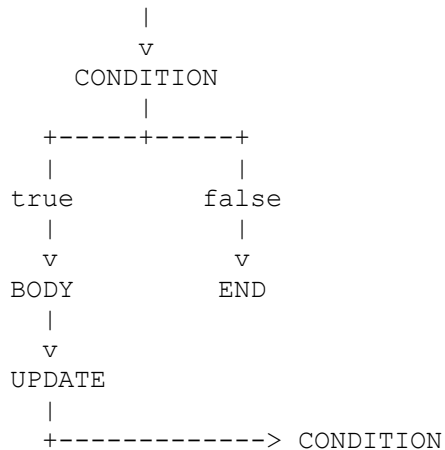
    System.out.println(i);
}
}

```

18. Key Concept to Remember

The for loop follows this exact basic sequence:

INITIALIZATION

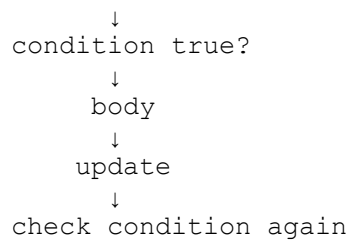


Core definition

A for loop is a Java iteration statement that repeatedly executes a block of code by performing initialization, checking a condition, executing the loop body when the condition is true, and then performing an update before checking the condition again.

The most important formula is:

```
for (initialization; condition; update)
```



In short:

Initialize → Check → Execute → Update → Check → Repeat → Exit.

Java 1.0 — while Loop

Java Version: 1.0

Year: 1996

Feature: while loop

Unique Class Name: WhileLoop

1. Definition

The **while loop** is an **iteration/repetition statement** in Java that repeatedly executes a block of code **as long as a specified boolean condition remains true**.

Basic syntax

```
while (condition) {  
    // statements  
}
```

Example:

```
int i = 1;
```

```
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

Output:

```
1  
2  
3  
4  
5
```

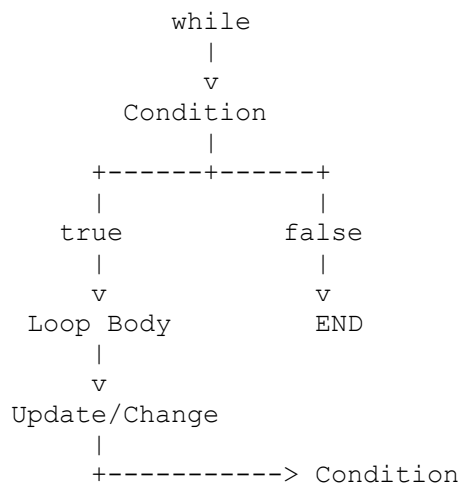
The while loop is an **entry-controlled loop** because Java checks the condition **before** executing the loop body.

2. Components of a while Loop

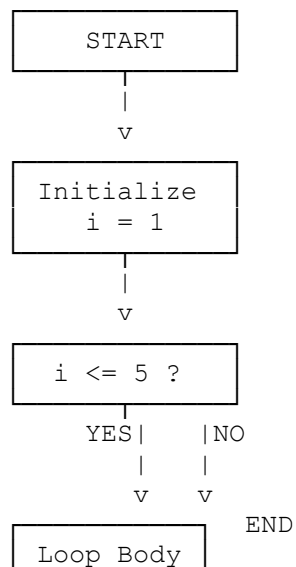
```
while (condition) {  
    statement;  
}
```

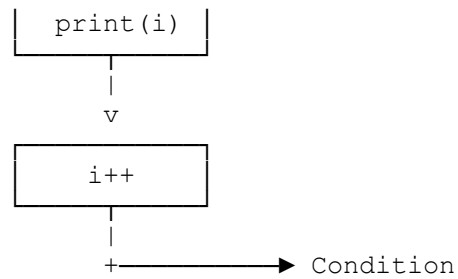
Component	Purpose
while	Java keyword that starts the loop
condition	Boolean expression controlling repetition
{ }	Loop body
statement	Code executed on each iteration

Conceptually:

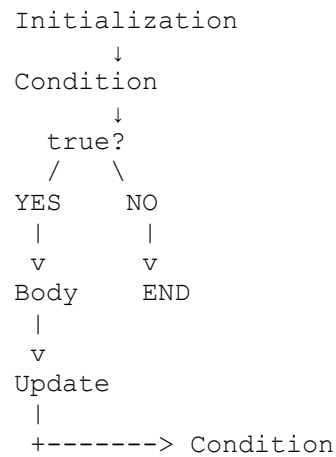


3. Flowchart





The essential flow is:



4. Java Code Example

```

public class WhileLoop {

    public static void main(String[] args) {

        int i = 1;

        while (i <= 5) {
            System.out.println("Iteration: " + i);
            i++;
        }
    }
}
  
```

Output

```

Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
  
```

5. Step-by-Step Execution

Consider:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Step 1 — Initialization

Before the loop:

```
int i = 1;
```

So:

```
i = 1
```

Step 2 — Condition

Java checks:

```
i <= 5
```

Initially:

```
1 <= 5 → true
```

Therefore, the body executes.

Step 3 — Execute Body

```
System.out.println(i);
```

Output:

```
1
```

Step 4 — Update

i++;

Now:

i = 2

Step 5 — Check Again

2 <= 5 → true

The loop repeats.

Eventually:

5 <= 5 → true

The body prints 5.

Then:

i++

makes:

i = 6

Now:

6 <= 5 → false

The loop terminates.

6. Complete Iteration Table

Iteration	i before condition	Condition	Body executes?	Output	i after i++
1	1	1 <= 5 → true	Yes	1	2
2	2	2 <= 5 → true	Yes	2	3
3	3	3 <= 5 → true	Yes	3	4
4	4	4 <= 5 → true	Yes	4	5

5	5	5 <= 5 → true	Yes	5	6
6	6	6 <= 5 → false	No	—	—

7. Important Characteristic: Zero Executions Are Possible

Because while checks its condition **before** entering the body, the body may execute **zero times**.

Example:

```
public class WhileLoop {
    public static void main(String[] args) {
        int i = 10;

        while (i <= 5) {
            System.out.println(i);
            i++;
        }

        System.out.println("Loop finished.");
    }
}
```

Output:

Loop finished.

Why?

```
i = 10
|
v
10 <= 5?
|
FALSE
|
v
END
```

The loop body never executes.

8. while Loop With Multiple Conditions

Logical operators can be used in the condition.

```
public class WhileLoop {  
    public static void main(String[] args) {  
        int i = 1;  
        int j = 10;  
  
        while (i <= 5 && j >= 6) {  
            System.out.println(  
                "i = " + i + ", j = " + j  
            );  
            i++;  
            j--;  
        }  
    }  
}
```

Output:

```
i = 1, j = 10  
i = 2, j = 9  
i = 3, j = 8  
i = 4, j = 7  
i = 5, j = 6
```

The loop continues only while **both** conditions are true.

9. while Loop With if

A while loop can contain conditional statements.

```
public class WhileLoop {  
    public static void main(String[] args) {  
        int i = 1;  
  
        while (i <= 10) {  
            if (i % 2 == 0) {  
                System.out.println(i + " is even");  
            }  
            i++;  
        }  
    }  
}
```

```

        i++;
    }
}

```

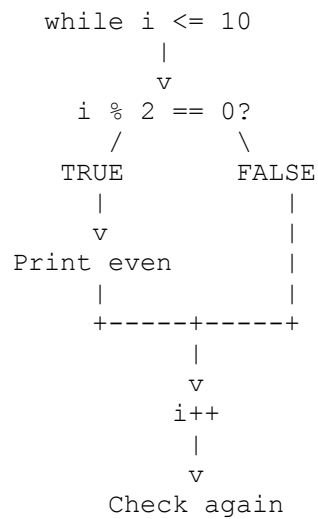
Output:

```

2  is even
4  is even
6  is even
8  is even
10 is even

```

Flow:



10. Infinite while Loop

If the condition never becomes false, the loop can continue indefinitely.

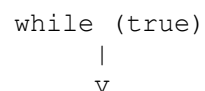
Example:

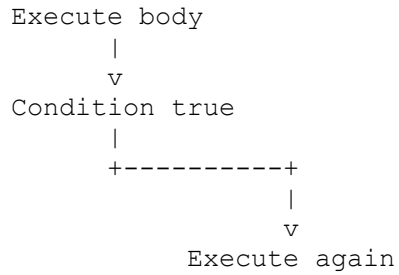
```

while (true) {
    System.out.println("Running...");
}

```

Flow:





Such a loop is commonly called an **infinite loop**.

It can be terminated using break:

```
public class WhileLoop {  
    public static void main(String[] args) {  
        int i = 1;  
        while (true) {  
            System.out.println(i);  
            if (i == 5) {  
                break;  
            }  
            i++;  
        }  
    }  
}
```

Output:

```
1  
2  
3  
4  
5
```

11. while Loop With break

break immediately terminates the loop.

```
public class WhileLoop {
```

```

public static void main(String[] args) {

    int i = 1;

    while (i <= 10) {

        if (i == 6) {
            break;
        }

        System.out.println(i);
        i++;
    }
}

```

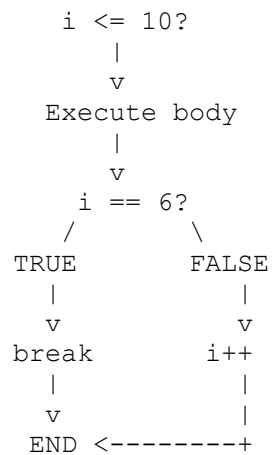
Output:

```

1
2
3
4
5

```

Flow:



12. while Loop With continue

continue skips the remaining part of the current iteration and starts the next iteration.

Example:

```

public class WhileLoop {

```

```

public static void main(String[] args) {

    int i = 0;

    while (i < 5) {

        i++;

        if (i == 3) {
            continue;
        }

        System.out.println(i);
    }
}

```

Output:

```

1
2
4
5

```

Notice that `i++` occurs **before** `continue`. This is important because otherwise the loop could become infinite.

13. Nested while Loops

A while loop can contain another while loop.

```

public class WhileLoop {

    public static void main(String[] args) {

        int i = 1;

        while (i <= 3) {

            int j = 1;

            while (j <= 3) {

                System.out.println(
                    "i = " + i + ", j = " + j
                );

                j++;
            }

            i++;
        }
    }
}

```

```

        }
        i++;
    }
}

```

Output:

```

i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 2, j = 2
i = 2, j = 3
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3

```

Conceptually:

```

Outer while
  |
  v
i = 1
  |
  v
Inner while
  |
  +-- j = 1
  +-- j = 2
  +-- j = 3
  |
  v
i++
  |
  v
Outer while again

```

14. while vs for

Both can perform repeated execution.

for

```

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}

```

Equivalent while

```
int i = 1;
```

```
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

The execution logic is essentially:

for:

Initialization → Condition → Body → Update → Condition

while:

Initialization → Condition → Body → Update → Condition

The primary difference is **where the loop-control elements are written**.

Feature	for	while
Initialization	Usually in loop header	Usually before loop
Condition	Loop header	Loop header
Update	Usually loop header	Usually body
Best suited	Counter-controlled loops	Condition-controlled loops
Minimum executions	0	0
Type	Entry-controlled	Entry-controlled

15. while vs do-while

This is an important distinction.

while

```
while (condition) {  
    // body  
}
```

Condition is checked **first**.

do-while

```
do {  
    // body  
} while (condition);
```

Body executes **first**, then condition is checked.

Example:

```
int i = 10;  
  
while (i < 5) {  
    System.out.println("while");  
}
```

Output:

Nothing

But:

```
int i = 10;  
  
do {  
    System.out.println("do-while");  
} while (i < 5);
```

Output:

do-while

Therefore:

while:
Condition → Body

do-while:
Body → Condition

16. Complete Java Code Example

```

public class WhileLoop {

    public static void main(String[] args) {

        // 1. Basic while loop
        System.out.println("=== Basic While Loop ===");

        int i = 1;

        while (i <= 5) {
            System.out.println("Iteration: " + i);
            i++;
        }

        // 2. Even numbers
        System.out.println("\n=== Even Numbers ===");

        int number = 1;

        while (number <= 10) {

            if (number % 2 == 0) {
                System.out.println(number);
            }

            number++;
        }

        // 3. Break
        System.out.println("\n=== Break ===");

        int count = 1;

        while (count <= 10) {

            if (count == 6) {
                break;
            }

            System.out.println(count);
            count++;
        }

        // 4. Continue
        System.out.println("\n=== Continue ===");

        int value = 0;

        while (value < 5) {

            value++;

            if (value == 3) {
                continue;
            }
        }
    }
}

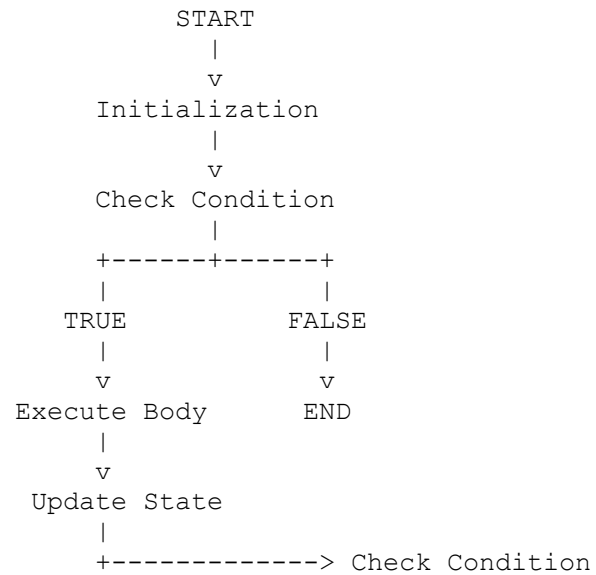
```

```

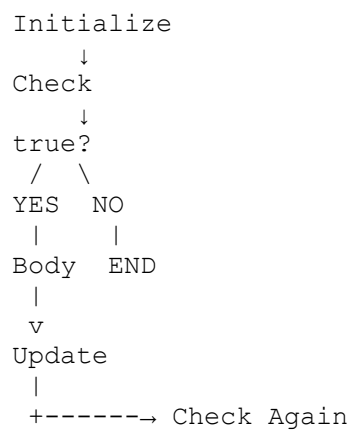
        }
        System.out.println(value);
    }
}

```

17. Overall while Loop Execution



Core formula



Key Definition

A while loop is a Java entry-controlled iteration statement that repeatedly executes a block of code while its boolean condition evaluates to true.

The most important sequence to remember is:

Initialize → Check → Execute → Update → Check → Repeat → Exit

And the most important characteristic is:

Because the condition is checked before the body, a while loop can execute zero or more times.

Java 1.0 — do-while Loop

Java Version: 1.0

Year: 1996

Feature: do-while loop

Unique Class Name: DoWhileLoop

1. Definition

The **do-while loop** is an **iteration/repetition statement** in Java that executes a block of code **at least once** and then repeatedly executes it as long as a specified boolean condition remains true.

Unlike the while loop, the do-while loop checks its condition **after** executing the loop body.

Basic syntax

```
do {  
    // statements  
} while (condition);
```

Important: There is a semicolon ; after the while (condition).

Simple example

```
int i = 1;
```

```
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

Output:

1
2
3
4
5

2. Main Components

```
do {  
    statements;  
} while (condition);
```

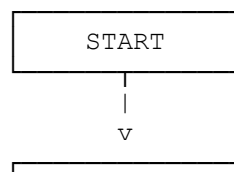
Component	Purpose
do	Starts the loop body
{ }	Contains statements to execute
while	Introduces the continuation condition
condition	Determines whether another iteration occurs
;	Terminates the do-while statement

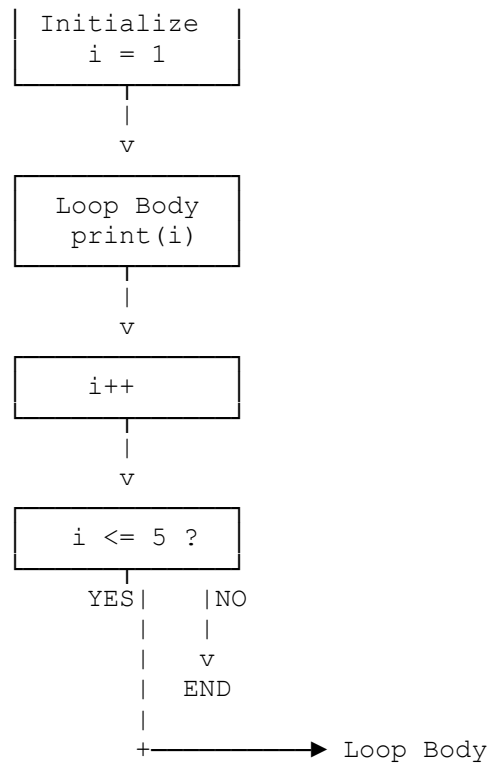
The fundamental difference is:

while:
Condition → Body

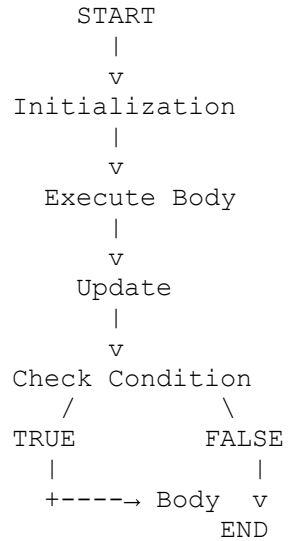
do-while:
Body → Condition

3. Flowchart





Conceptual flow



4. Java Code Example

```
public class DoWhileLoop {
```

```
public static void main(String[] args) {  
  
    int i = 1;  
  
    do {  
        System.out.println("Iteration: " + i);  
        i++;  
    } while (i <= 5);  
}  
}
```

Output

```
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4  
Iteration: 5
```

5. Step-by-Step Execution

Consider:

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

Step 1 — Initialization

```
int i = 1;  
i = 1
```

Step 2 — Execute the body

The do block executes **without checking the condition first**.

```
System.out.println(i);
```

Output:

1

Step 3 — Update

`i++;`

Now:

`i = 2`

Step 4 — Check condition

`i <= 5`

`2 <= 5 → true`

Therefore, the body executes again.

This continues until:

`i = 6`

Then:

`6 <= 5 → false`

The loop terminates.

6. Complete Iteration Table

Iteration	i before body	Body executes?	Output	i after i++	Condition
1	1	Yes	1	2	<code>2 <= 5 → true</code>
2	2	Yes	2	3	<code>3 <= 5 → true</code>
3	3	Yes	3	4	<code>4 <= 5 → true</code>
4	4	Yes	4	5	<code>5 <= 5 → true</code>
5	5	Yes	5	6	<code>6 <= 5 → false</code>

7. Most Important Feature: Executes at Least Once

This is the most important characteristic of do-while.

Consider:

```
int i = 10;
```

```
do {  
    System.out.println(i);  
} while (i <= 5);
```

Output:

10

Why?

Because Java executes:

```
do {  
    System.out.println(i);  
}
```

before evaluating:

```
i <= 5
```

The flow is:

```
i = 10  
  |  
  v  
Execute body  
  |  
  v  
Print 10  
  |  
  v  
10 <= 5 ?  
  |  
FALSE  
  |  
  v  
END
```

Therefore:

A do-while loop always executes its body at least once.

8. Comparison With while

This is the most important comparison.

while

```
int i = 10;
```

```
while (i <= 5) {  
    System.out.println(i);  
}
```

Output:

Nothing

Because:

$10 \leq 5 \rightarrow \text{false}$

The body is never executed.

do-while

```
int i = 10;
```

```
do {  
    System.out.println(i);  
} while (i <= 5);
```

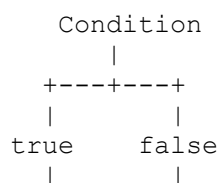
Output:

10

Because the body executes before the condition is checked.

Comparison

while:



```

      v      v
Body    END

```

do-while:

```

      Body
      |
      v
    Condition
   /        \
 true        false
  |           |
+---> Body END

```

9. do-while With if

A do-while loop can contain conditional statements.

```

public class DoWhileLoop {

    public static void main(String[] args) {

        int i = 1;

        do {

            if (i % 2 == 0) {
                System.out.println(i + " is even");
            }

            i++;

        } while (i <= 10);

    }
}

```

Output:

```

2  is even
4  is even
6  is even
8  is even
10 is even

```

10. do-while With break

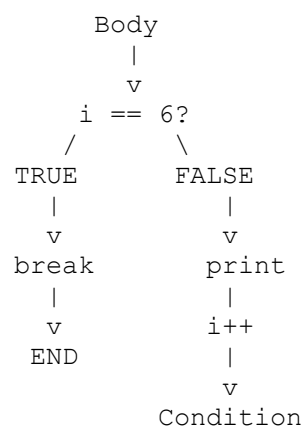
break immediately terminates the loop.

```
public class DoWhileLoop {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            if (i == 6) {  
                break;  
            }  
            System.out.println(i);  
            i++;  
        } while (i <= 10);  
    }  
}
```

Output:

1
2
3
4
5

Flow:



11. do-while With continue

continue causes the loop to proceed toward its condition check.

Example:

```
public class DoWhileLoop {  
    public static void main(String[] args) {  
        int i = 0;  
        do {  
            i++;  
            if (i == 3) {  
                continue;  
            }  
            System.out.println(i);  
        } while (i < 5);  
    }  
}
```

Output:

```
1  
2  
4  
5
```

When `i == 3`, the `continue` skips the remaining statements in that iteration and proceeds to the `while (i < 5)` condition.

12. Nested do-while Loops

A do-while loop can contain another do-while loop.

```
public class DoWhileLoop {  
    public static void main(String[] args) {  
        int i = 1;  
        do {
```

```

        int j = 1;

        do {
            System.out.println(
                "i = " + i + ", j = " + j
            );

            j++;

        } while (j <= 3);

        i++;

    } while (i <= 3);
}

```

Output:

```

i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1
i = 2, j = 2
i = 2, j = 3
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3

```

13. Practical Example — Menu

A do-while loop is particularly useful for menus because the menu should normally be displayed **at least once**.

```

public class DoWhileLoop {

    public static void main(String[] args) {

        int choice = 1;

        do {

            System.out.println("==== MENU =====");
            System.out.println("1. Create User");
            System.out.println("2. Update User");
            System.out.println("3. Delete User");
            System.out.println("4. Exit");

            System.out.println("Selected option: " + choice);

```

```

        choice++;
    } while (choice <= 4);
}

```

Output:

```

===== MENU =====
1. Create User
2. Update User
3. Delete User
4. Exit
Selected option: 1

===== MENU =====
1. Create User
2. Update User
3. Delete User
4. Exit
Selected option: 2

...

```

In real applications, a Scanner or another input mechanism would normally be used to obtain the user's choice.

14. while vs do-while vs for

Feature	while	do-while	for
Type	Entry-controlled	Exit-controlled	Entry-controlled
Condition checked	Before body	After body	Before body
Minimum executions	0	1	0
Initialization	Usually before loop	Usually before loop	Usually in header
Update	Usually in body	Usually in body	Usually in header
Good for	Unknown repetition	At-least-once operations	Counter-controlled repetition

Visual comparison

```
while
-----
Condition → Body → Condition → Body → ... → END
```

```
do-while
-----
Body → Condition → Body → Condition → ... → END
```

```
for
---
Initialization → Condition → Body → Update
                    ↑           |
                    +-----+
```

15. Complete Java Code Example

```
public class DoWhileLoop {

    public static void main(String[] args) {

        // 1. Basic do-while loop
        System.out.println("=== Basic do-while Loop ===");

        int i = 1;

        do {
            System.out.println("Iteration: " + i);
            i++;
        } while (i <= 5);

        // 2. Demonstrating at-least-once execution
        System.out.println("\n=== At Least Once ===");

        int value = 10;

        do {
            System.out.println("Value: " + value);
            value++;
        } while (value <= 5);

        // 3. Even numbers
        System.out.println("\n=== Even Numbers ===");

        int number = 1;

        do {

            if (number % 2 == 0) {
```

```

        System.out.println(number);
    }

    number++;

} while (number <= 10);

// 4. Break
System.out.println("\n=== Break ===");

int count = 1;

do {

    if (count == 6) {
        break;
    }

    System.out.println(count);
    count++;

} while (count <= 10);

// 5. Continue
System.out.println("\n=== Continue ===");

int n = 0;

do {

    n++;

    if (n == 3) {
        continue;
    }

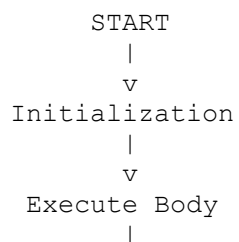
    System.out.println(n);

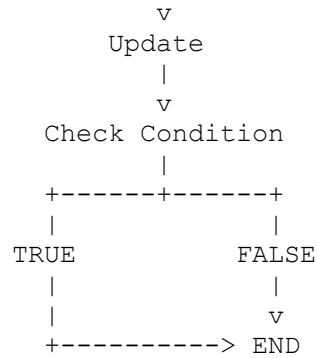
} while (n < 5);

}

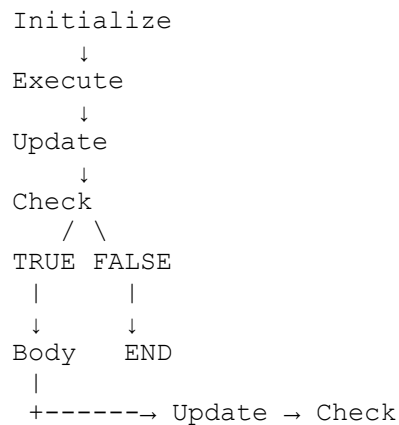
```

16. Overall Execution Flow





Core formula



Key Definition

A do-while loop is a Java exit-controlled iteration statement that executes its body first and evaluates its boolean condition afterward; therefore, its body executes at least once.

The most important sequence to remember is:

Initialize → Execute → Update → Check → Repeat → Exit

And the key distinction is:

while checks first; do-while executes first and checks afterward.

Java 1.0 — break / continue

Java Version: 1.0

Year: 1996

Feature: break / continue

Unique Class Name: BreakContinue

break and continue are **loop-control statements**. They change the normal flow of execution inside loops such as for, while, and do-while.

1. Definition

break

The **break statement** immediately terminates the **nearest enclosing loop or switch statement** and transfers control to the statement following it.

Syntax

break;

Example:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4
```

When `i == 5`, break terminates the loop.

continue

The **continue statement** skips the remaining statements of the **current iteration** of the nearest enclosing loop and proceeds with the loop's next iteration.

Syntax

```
continue;
```

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Output:

```
1  
2  
4  
5
```

When `i == 3`, the current iteration is skipped.

2. Fundamental Difference

The easiest way to remember them is:

```
break  
↓  
STOP THE LOOP
```

```
continue  
↓  
SKIP THIS ITERATION  
↓  
GO TO NEXT ITERATION
```

Statement	What it does	Result
-----------	--------------	--------

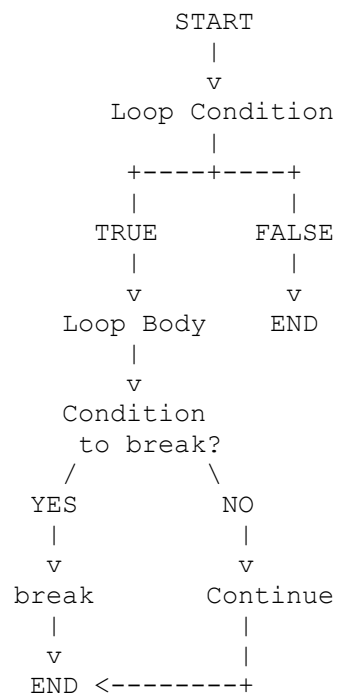
break	Terminates the loop	Exits loop completely
continue	Skips current iteration	Continues with next iteration

Simple analogy

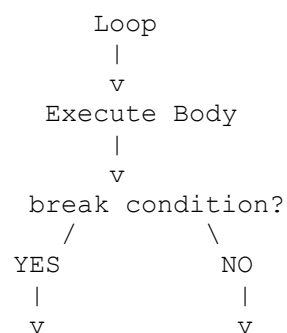
break = "I'm done with this loop."

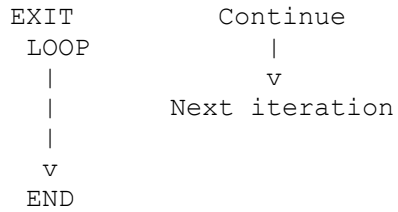
continue = "Skip this one and continue."

3. break Flowchart



A more precise flow is:

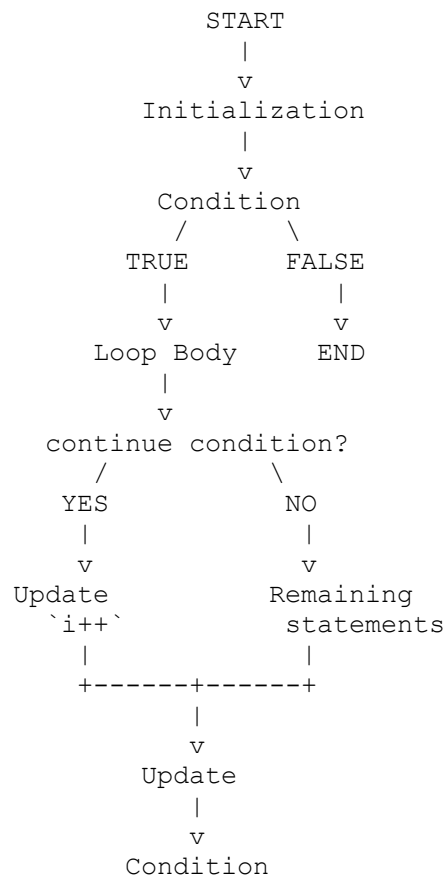




4. continue Flowchart

The exact destination of continue depends on the type of loop.

For a `for` loop:



The important point is that in a **for loop**, `continue` does **not** simply jump directly to the condition; it proceeds to the loop's **update expression first**.

For example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

When i == 3:

```
continue  
↓  
i++  
↓  
condition  
↓  
next iteration
```

5. Complete break Example

```
public class BreakContinue {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 6) {  
                break;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Output

```
1  
2  
3  
4  
5
```

Execution

```
i = 1 → print 1
i = 2 → print 2
i = 3 → print 3
i = 4 → print 4
i = 5 → print 5
i = 6 → break
      ↓
    EXIT
```

Notice that values 6 through 10 are never processed by the loop after the break.

6. Complete continue Example

```
public class BreakContinue {

    public static void main(String[] args) {

        for (int i = 1; i <= 10; i++) {

            if (i == 6) {
                continue;
            }

            System.out.println(i);
        }
    }
}
```

Output

```
1
2
3
4
5
7
8
9
10
```

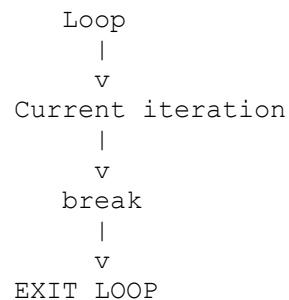
Here, only the iteration where `i == 6` is skipped.

```
i = 1 → print
i = 2 → print
i = 3 → print
```

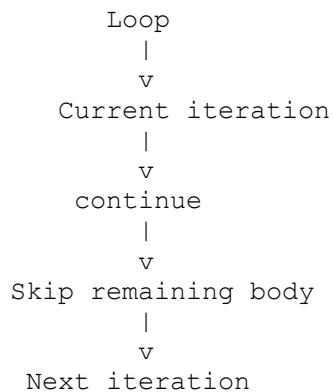
i = 4 → print
i = 5 → print
i = 6 → continue → skip
i = 7 → print
i = 8 → print
i = 9 → print
i = 10 → print

7. break vs continue Flow

break



continue



8. break With while

break can be used inside a while loop.

```
public class BreakContinue {
```

```

public static void main(String[] args) {

    int i = 1;

    while (i <= 10) {

        if (i == 6) {
            break;
        }

        System.out.println(i);
        i++;
    }
}

```

Output:

```

1
2
3
4
5

```

Flow:

```

i <= 10?
  |
  v
Execute body
  |
  v
i == 6?
 /      \
YES       NO
 |         |
 v         v
break     print
 |         |
 v         i++
END        |
          +-----> condition

```

9. continue With while

Example:

```

public class BreakContinue {

```

```

public static void main(String[] args) {

    int i = 0;

    while (i < 5) {

        i++;

        if (i == 3) {
            continue;
        }

        System.out.println(i);
    }
}

```

Output:

```

1
2
4
5

```

Important

With a while loop, be careful about the loop-control variable.

This can create an infinite loop:

```

int i = 0;

while (i < 5) {

    if (i == 3) {
        continue;
    }

    i++;
}

```

When i becomes 3, continue executes **before i++**, so i remains 3 forever.

A safer version is:

```

int i = 0;

while (i < 5) {

```

```
i++;  
  
if (i == 3) {  
    continue;  
}  
  
System.out.println(i);  
}
```

10. break With do-while

```
public class BreakContinue {  
  
    public static void main(String[] args) {  
  
        int i = 1;  
  
        do {  
  
            if (i == 4) {  
                break;  
            }  
  
            System.out.println(i);  
            i++;  
  
        } while (i <= 10);  
    }  
}
```

Output:

1
2
3

When i == 4:

break
↓
Exit do-while
↓
Continue after loop

11. continue With do-while

```
public class BreakContinue {  
    public static void main(String[] args) {  
        int i = 0;  
        do {  
            i++;  
            if (i == 3) {  
                continue;  
            }  
            System.out.println(i);  
        } while (i < 5);  
    }  
}
```

Output:

1
2
4
5

For a do-while loop, continue proceeds to the **condition check**.

```
Body  
↓  
continue  
↓  
while(condition)  
↓  
true?  
↓  
Body again
```

12. break Inside switch

break is not limited to loops. It can also terminate a switch statement.

```
public class BreakContinue {  
    public static void main(String[] args) {  
        int option = 2;  
        switch (option) {  
            case 1:  
                System.out.println("Create");  
                break;  
            case 2:  
                System.out.println("Update");  
                break;  
            case 3:  
                System.out.println("Delete");  
                break;  
            default:  
                System.out.println("Invalid");  
        }  
    }  
}
```

Output:

Update

Here:

case 2
↓
Print Update
↓
break
↓
Exit switch

Important distinction

continue is a **loop-control statement** and is not used to directly continue a switch by itself.

13. Nested Loops and break

An important rule:

An ordinary break terminates only the nearest enclosing loop or switch.

Example:

```
public class BreakContinue {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (j == 2) {  
                    break;  
                }  
                System.out.println(  
                    "i = " + i + ", j = " + j  
                );  
            }  
        }  
    }  
}
```

Output:

```
i = 1, j = 1  
i = 2, j = 1  
i = 3, j = 1
```

The inner loop is terminated, but the outer loop continues.

```
Outer Loop  
|  
+-- Inner Loop  
|   |  
|   +-- j = 1 → print  
|   |  
|   +-- j = 2 → break  
|  
+-- Outer continues  
|  
+-- Inner Loop again
```

14. Nested Loops and continue

Similarly, an ordinary continue affects the **nearest enclosing loop**.

```
public class BreakContinue {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (j == 2) {  
                    continue;  
                }  
  
                System.out.println(  
                    "i = " + i + ", j = " + j  
                );  
            }  
        }  
    }  
}
```

Output:

```
i = 1, j = 1  
i = 1, j = 3  
i = 2, j = 1  
i = 2, j = 3  
i = 3, j = 1  
i = 3, j = 3
```

Only the inner-loop iteration where `j == 2` is skipped.

15. Labeled break

Java also supports a **labeled break**, which can terminate an outer loop.

Syntax:

```
label:  
for (...) {  
    ...  
    break label;  
}
```

Example:

```

public class BreakContinue {

    public static void main(String[] args) {

        outer:
        for (int i = 1; i <= 3; i++) {

            for (int j = 1; j <= 3; j++) {

                if (i == 2 && j == 2) {
                    break outer;
                }

                System.out.println(
                    "i = " + i + ", j = " + j
                );
            }
        }
    }
}

```

Output:

```

i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 2, j = 1

```

When:

```

i = 2
j = 2

```

this executes:

```
break outer;
```

and terminates the **outer loop**.

Flow:

```

    outer:
Outer Loop
    |
    v
  Inner Loop
    |
    v
i == 2 && j == 2?
  /       \

```

YES	NO
v	v
break outer	continue
v	
EXIT OUTER LOOP	

16. Labeled continue

Java also supports a **labeled continue**, which causes the next iteration of a specified enclosing loop.

Example:

```
public class BreakContinue {

    public static void main(String[] args) {

        outer:
        for (int i = 1; i <= 3; i++) {

            for (int j = 1; j <= 3; j++) {

                if (j == 2) {
                    continue outer;
                }

                System.out.println(
                    "i = " + i + ", j = " + j
                );
            }
        }
    }
}
```

Output:

```
i = 1, j = 1
i = 2, j = 1
i = 3, j = 1
```

When j == 2:

```
continue outer
↓
Skip remainder of inner loop
↓
```

Next iteration of outer loop

This is different from ordinary:

`continue;`

which affects the nearest enclosing loop.

17. break and continue Comparison

Feature	break	continue
Purpose	Terminates loop	Skips current iteration
Loop continues?	No	Yes
Current iteration	Terminates	Skipped
Next iteration	Does not occur	Occurs if condition permits
Can be used in loops	Yes	Yes
Can be used in switch	Yes	No
Labeled form	break label;	continue label;
Typical use	Stop when target found	Ignore unwanted values

18. Real-World Example — Searching

break is useful when searching for an item.

```
public class BreakContinue {  
  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        int target = 30;  
        boolean found = false;
```

```

    for (int number : numbers) {

        if (number == target) {
            found = true;
            break;
        }

    }

    if (found) {
        System.out.println("Number found");
    } else {
        System.out.println("Number not found");
    }
}

```

Output:

Number found

Once 30 is found, there is no reason to continue searching.

```

10 → not found
20 → not found
30 → FOUND
      ↓
    break
      ↓
    EXIT

```

19. Real-World Example — Skip Invalid Values

continue is useful when some values should be ignored.

```

public class BreakContinue {

    public static void main(String[] args) {

        int[] numbers = {
            10, -5, 20, -10, 30
        };

        for (int number : numbers) {

            if (number < 0) {
                continue;
            }

```

```

        }
        System.out.println(number);
    }
}
}

```

Output:

```

10
20
30

```

The negative values are skipped.

```

10      → process
-5      → continue → skip
20      → process
-10     → continue → skip
30      → process

```

20. Complete Java Code Example

```

public class BreakContinue {

    public static void main(String[] args) {

        // =====
        // 1. BREAK
        // =====

        System.out.println("=== break ===");

        for (int i = 1; i <= 10; i++) {

            if (i == 6) {
                break;
            }

            System.out.println(i);
        }

        // =====
        // 2. CONTINUE
        // =====
    }
}

```

```

System.out.println("\n=== continue ===");

for (int i = 1; i <= 10; i++) {

    if (i == 6) {
        continue;
    }

    System.out.println(i);
}

// =====
// 3. BREAK WITH SEARCH
// =====

System.out.println("\n=== Search ===");

int[] numbers = {10, 20, 30, 40, 50};

int target = 30;

for (int number : numbers) {

    if (number == target) {
        System.out.println("Found: " + number);
        break;
    }

}

// =====
// 4. CONTINUE WITH FILTERING
// =====

System.out.println("\n=== Skip Negative Numbers ===");

int[] values = {
    10, -5, 20, -10, 30
};

for (int value : values) {

    if (value < 0) {
        continue;
    }

    System.out.println(value);
}

// =====
// 5. LABELED BREAK
// =====

System.out.println("\n=== Labeled break ===");

```

```

outer:
for (int i = 1; i <= 3; i++) {

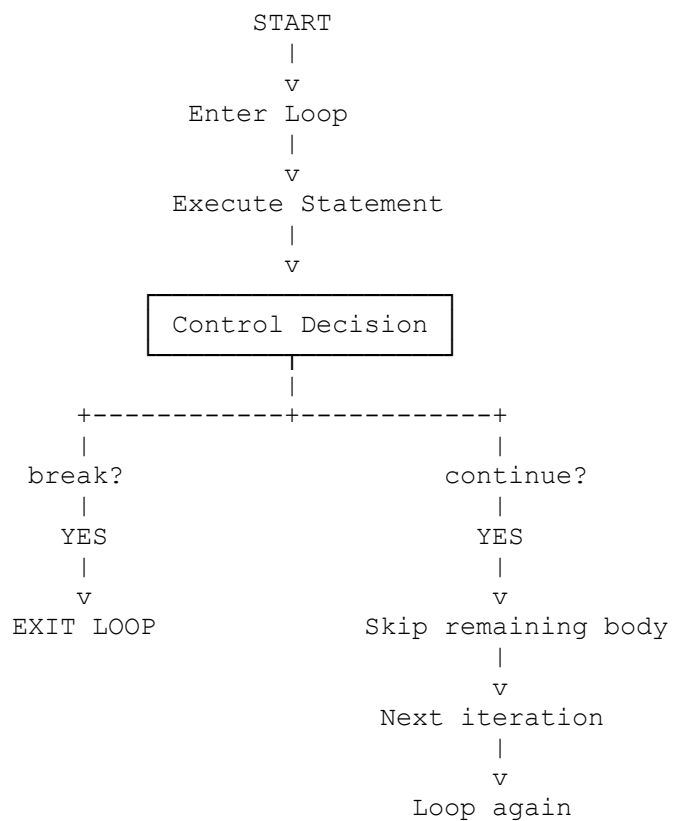
    for (int j = 1; j <= 3; j++) {

        if (i == 2 && j == 2) {
            break outer;
        }

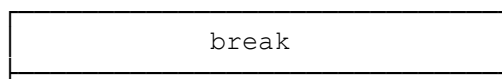
        System.out.println(
            "i = " + i + ", j = " + j
        );
    }
}
}

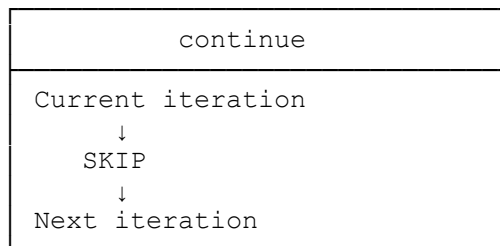
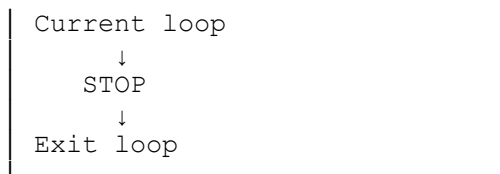
```

21. Overall Flowchart



The conceptual difference is:





22. Key Definitions

break

break is a Java control-flow statement that immediately terminates the nearest enclosing loop or switch statement and transfers control to the statement following it.

continue

continue is a Java control-flow statement that skips the remaining statements of the current iteration of the nearest enclosing loop and proceeds toward the next iteration.

Easy memory rule

break
= EXIT

continue
= SKIP + NEXT ITERATION

So the complete concept is:

break → stop the loop completely

continue → skip the current iteration and keep looping

Java 1.0 — Methods

Java Version: 1.0

Year: 1996

Feature: Methods

Unique Class Name: Methods

1. Definition

A **method** in Java is a named block of code defined inside a class that performs a specific task. A method can accept **parameters**, execute statements, and optionally **return a value** to its caller.

Methods are fundamental to Java because they provide:

- **Code reuse**
- **Modularity**
- **Abstraction**
- **Readability**
- **Maintainability**
- **Separation of responsibilities**

Basic syntax

```
accessModifier returnType methodName(parameterList) {  
    // method body  
    return value;  
}
```

Example:

```
public static int add(int a, int b) {  
    return a + b;  
}
```

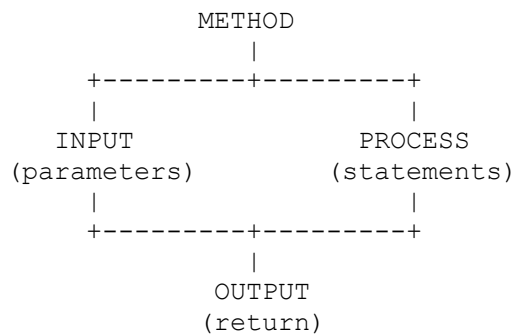
Here:

Part	Example	Meaning
Access modifier	public	Controls accessibility
Modifier	static	Method belongs to the class
Return type	int	Type of value returned
Method name	add	Name used to invoke the method
Parameters	int a, int b	Input values

Method body	{ return a + b; }	Instructions performed
return	return a + b	Sends result to caller

2. Method Concept

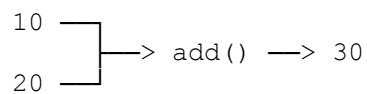
A method can be viewed as a small reusable program inside a class:



For example:

```
int result = add(10, 20);
```

The flow is:



3. Why Methods Are Used

Without methods, code can become repetitive:

```
System.out.println("Hello");
System.out.println("Hello");
System.out.println("Hello");
```

With a method:

```
static void sayHello() {
```

```
    System.out.println("Hello");  
}
```

It can be called repeatedly:

```
sayHello();  
sayHello();  
sayHello();
```

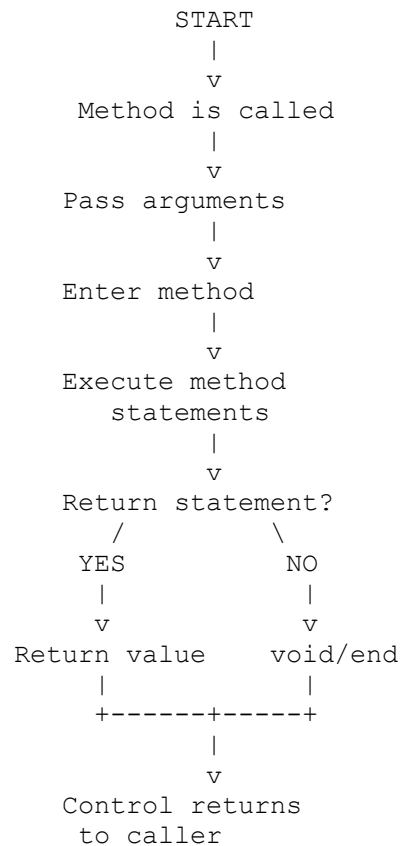
Output:

```
Hello  
Hello  
Hello
```

The method allows the same behavior to be defined once and reused.

4. Method Flowchart

A general method invocation can be represented as:



|
v
END

5. Complete Method Flow

Consider:

```
int result = add(10, 20);
```

and:

```
static int add(int a, int b) {  
    return a + b;  
}
```

The execution flow is:

```
main()  
  |  
  v  
add(10, 20)  
  |  
  v  
a = 10, b = 20  
  |  
  v  
a + b = 30  
  |  
  v  
return 30  
  |  
  v  
result = 30  
  |  
  v  
Continue main()
```

6. Java Code Example

```
public class Methods {  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}
```

```
public static void main(String[] args) {  
    int result = add(10, 20);  
    System.out.println("Result = " + result);  
}  
}
```

Output

Result = 30

7. Step-by-Step Execution

Step 1 — Program starts

```
public static void main(String[] args)
```

The JVM invokes main().

Step 2 — add() is called

```
int result = add(10, 20);
```

Arguments:

10
20

are passed to:

```
int a  
int b
```

So:

```
a = 10  
b = 20
```

Step 3 — Method body executes

```
return a + b;
```

Therefore:

$10 + 20 = 30$

Step 4 — Value is returned

```
return 30;
```

Step 5 — Caller receives the result

```
int result = 30;
```

Step 6 — Result is printed

```
Result = 30
```

8. Method With No Parameters and No Return Value

A method does not have to accept parameters or return a value.

```
public class Methods {  
    public static void greet() {  
        System.out.println("Hello, Java!");  
    }  
    public static void main(String[] args) {  
        greet();  
    }  
}
```

Output:

Hello, Java!

The return type is:

void

void means the method does not return a value to its caller.

9. Method With Parameters

Parameters allow data to be passed into a method.

```
public class Methods {  
    public static void greet(String name) {  
        System.out.println("Hello, " + name);  
    }  
    public static void main(String[] args) {  
        greet("Anurag");  
    }  
}
```

Output:

Hello, Anurag

Flow:

```
"Anurag"  
  |  
  v  
greet(name)  
  |  
  v  
name = "Anurag"  
  |  
  v  
Print message
```

10. Method With Return Value

```
public class Methods {  
    public static int square(int number) {  
        return number * number;  
    }  
    public static void main(String[] args) {
```

```

        int result = square(5);

        System.out.println("Square = " + result);
    }
}

```

Output:

Square = 25

Flow:

```

square(5)
  |
  v
number = 5
  |
  v
5 * 5
  |
  v
25
  |
  v
return 25

```

11. Method With Multiple Parameters

```

public class Methods {

    public static int multiply(int a, int b, int c) {

        return a * b * c;
    }

    public static void main(String[] args) {

        int result = multiply(2, 3, 4);

        System.out.println(result);
    }
}

```

Output:

24

The method receives:

a = 2
b = 3
c = 4

and calculates:

$2 \times 3 \times 4 = 24$

12. Method With boolean Return Type

Methods can return any appropriate Java type.

```
public class Methods {  
    public static boolean isEven(int number) {  
        return number % 2 == 0;  
    }  
    public static void main(String[] args) {  
        boolean result = isEven(10);  
        System.out.println(result);  
    }  
}
```

Output:

true

13. Method With String Return Type

```
public class Methods {  
    public static String getMessage() {  
        return "Welcome to Java";  
    }  
}
```

```
}  
  
public static void main(String[] args) {  
    String message = getMessage();  
    System.out.println(message);  
}  
}
```

Output:

Welcome to Java

14. void Method

A void method performs an operation without returning a value.

```
public class Methods {  
    public static void printNumbers() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
  
    public static void main(String[] args) {  
        printNumbers();  
    }  
}
```

Output:

```
1  
2  
3  
4  
5
```

Conceptually:

```
printNumbers()  
  |  
  v  
Execute loop  
  |  
  v  
No return value  
  |  
  v  
Back to caller
```

15. Instance Method

Not every method has to be static.

An **instance method** belongs to an object of the class.

```
public class Methods {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public static void main(String[] args) {  
        Methods object = new Methods();  
        int result = object.add(10, 20);  
        System.out.println(result);  
    }  
}
```

Output:

30

Here:

```
Methods object = new Methods();
```

creates an object.

Then:

```
object.add(10, 20);
```

invokes the instance method.

16. Static Method

A **static method** belongs to the class rather than to a particular object.

```
public class Methods {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int result = Methods.add(10, 20);  
        System.out.println(result);  
    }  
}
```

Output:

30

Because add() is static, it can be invoked using the class name:

```
Methods.add(10, 20);
```

17. Method Invocation

Method invocation means calling/executing a method.

Method definition:

```
public static int add(int a, int b) {  
    return a + b;  
}
```

Method invocation:

```
add(10, 20);
```

The relationship is:

```
METHOD DEFINITION
    |
    v
add(int a, int b)
    |
    |
    v
METHOD INVOCATION
    |
    v
add(10, 20)
    |
    v
EXECUTION
    |
    v
return 30
```

18. Parameters vs Arguments

These two terms are often confused.

Parameter

A **parameter** is a variable declared in the method definition.

```
public static int add(int a, int b)
```

Here:

a and b = parameters

Argument

An **argument** is the actual value supplied when the method is called.

```
add(10, 20);
```

Here:

10 and 20 = arguments

Concept Example

Parameter int a

Parameter int b

Argument 10

Argument 20

19. Return Statement

The return statement transfers control from the method back to its caller.

Example:

```
public static int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

Flow:

```
Caller  
  |  
  v  
add(10, 20)  
  |  
  v  
result = 30  
  |  
  v  
return result  
  |  
  v  
30  
  |  
  v  
Caller
```

For a non-void method, the returned value must be compatible with the declared return type.

20. Early return

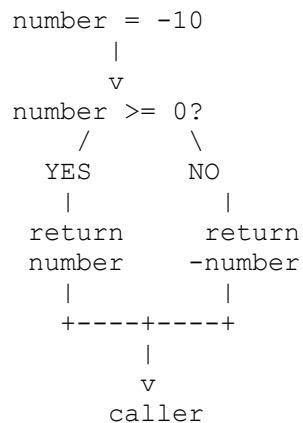
A method can return before reaching the end of its body.

```
public class Methods {  
    public static int absolute(int number) {  
        if (number >= 0) {  
            return number;  
        }  
        return -number;  
    }  
    public static void main(String[] args) {  
        System.out.println(absolute(-10));  
    }  
}
```

Output:

10

Flow:



21. Method Overloading

Java allows multiple methods with the **same name** when their parameter lists differ. This is called **method overloading**.

Example:

```
public class Methods {  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static double add(double a, double b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println(add(10, 20));  
  
        System.out.println(add(10, 20, 30));  
  
        System.out.println(add(10.5, 20.5));  
    }  
}
```

Output:

30
60
31.0

The compiler determines which overloaded method should be invoked based on the arguments.

add(10, 20)

↓

add(int, int)

add(10, 20, 30)

↓

add(int, int, int)

add(10.5, 20.5)

↓

add(double, double)

22. Recursive Method

A method can call itself. This is called **recursion**.

Example:

```
public class Methods {  
  
    public static int factorial(int n) {  
  
        if (n == 0) {  
            return 1;  
        }  
  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println(factorial(5));  
    }  
}
```

Output:

120

Execution:

```
factorial(5)  
  |  
  v  
5 × factorial(4)  
  |  
  v  
4 × factorial(3)  
  |  
  v  
3 × factorial(2)  
  |  
  v  
2 × factorial(1)  
  |  
  v  
1 × factorial(0)  
  |  
  v  
1
```

Then results return upward:

1
↓
1 × 1 = 1
↓
2 × 1 = 2
↓
3 × 2 = 6
↓
4 × 6 = 24
↓
5 × 24 = 120

23. Method Calling Another Method

Methods can call other methods.

```
public class Methods {  
  
    public static int square(int n) {  
        return n * n;  
    }  
  
    public static int calculate(int n) {  
        return square(n) + 10;  
    }  
  
    public static void main(String[] args) {  
  
        int result = calculate(5);  
  
        System.out.println(result);  
    }  
}
```

Output:

35

Flow:

```
main()  
|  
v  
calculate(5)  
|  
v  
square(5)
```

```
|  
v  
25  
|  
v  
25 + 10  
|  
v  
35  
|  
v  
main()
```

24. Complete Java Example

```
public class Methods {  
  
    // Method without parameters  
    public static void greet() {  
        System.out.println("Hello, Java!");  
    }  
  
    // Method with parameters  
    public static void greetUser(String name) {  
        System.out.println("Hello, " + name);  
    }  
  
    // Method with return value  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method with boolean return value  
    public static boolean isEven(int number) {  
        return number % 2 == 0;  
    }  
  
    // Method returning String  
    public static String getMessage() {  
        return "Welcome to Java Methods";  
    }  
  
    public static void main(String[] args) {  
  
        // Calling method without parameters  
        greet();  
  
        // Calling method with parameter  
        greetUser("Anurag");  
    }  
}
```

```

        // Calling method with return value
        int sum = add(10, 20);
        System.out.println("Sum = " + sum);

        // Calling boolean method
        boolean even = isEven(10);
        System.out.println("Is even = " + even);

        // Calling String-returning method
        String message = getMessage();
        System.out.println(message);
    }
}

```

Output

```

Hello, Java!
Hello, Anurag
Sum = 30
Is even = true
Welcome to Java Methods

```

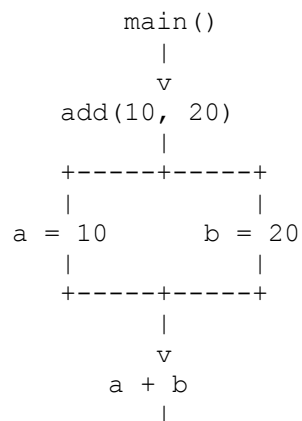
25. Method Execution Model

A method invocation creates a method execution context associated with the current thread's call stack.

For:

```
int result = add(10, 20);
```

the conceptual flow is:



```
      v
      30
      |
      v
return 30
      |
      v
result = 30
      |
      v
main() continues
```

26. Important Rules About Methods

Rule 1 — A method must be declared inside a class

Valid:

```
public class Methods {

    public static void test() {
        // ...
    }
}
```

Java does not use free-standing functions outside classes in the ordinary Java language model.

Rule 2 — Method name must be followed by parentheses

```
test();
```

The parentheses indicate a method invocation.

Rule 3 — A non-void method must return an appropriate value

Valid:

```
public static int add(int a, int b) {
    return a + b;
}
```

Invalid:

```
public static int add(int a, int b) {
    System.out.println(a + b);
}
```

because an int method does not return an int value.

Rule 4 — A void method does not need to return a value

```
public static void printHello() {
    System.out.println("Hello");
}
```

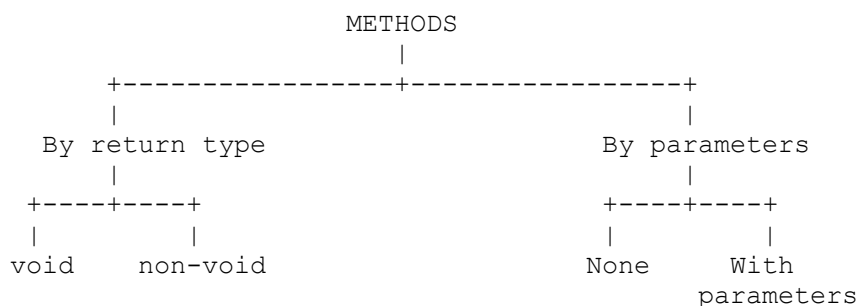
Rule 5 — Parameters are local to the method

```
public static int add(int a, int b) {
    return a + b;
}
```

The variables a and b belong to that method invocation.

27. Method Categories

Methods can be categorized in several ways:



They can also be:

```
Methods
|
+-- Static methods
|
+-- Instance methods
```

```
|  
+-- Overloaded methods  
|  
+-- Recursive methods
```

28. Method vs Constructor

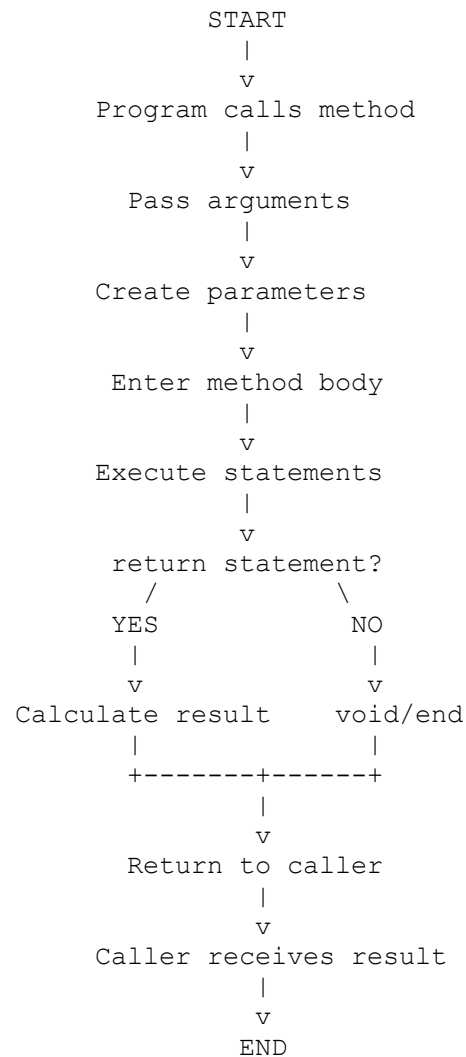
A method and a constructor are different concepts.

Method	Constructor
Performs an operation	Initializes an object
Has a return type, including void	Has no return type
Can have any valid name	Must have same name as class
Called explicitly	Invoked during object creation
Can be inherited/overridden depending on type	Not inherited/overridden as methods are
Example: add()	Example: Methods()

Example:

```
public class Methods {  
  
    // Constructor  
    public Methods() {  
        System.out.println("Object created");  
    }  
  
    // Method  
    public void display() {  
        System.out.println("Display method");  
    }  
}
```

29. Method Flowchart — Complete View



30. Key Definition

A method in Java is a named, reusable block of code declared within a class that performs a specific operation and may accept parameters and return a value to its caller.

Remember the fundamental structure:

```
METHOD
|
+-- Name
|
+-- Parameters
|
+-- Body
```

|
+-- Return type
|
+-- Return value (if applicable)

And the execution sequence is:

Define Method
↓
Call Method
↓
Pass Arguments
↓
Execute Method Body
↓
Return Value (if any)
↓
Continue Caller

Core idea:

Method = Define once → Call whenever needed → Execute → Return result (if applicable).